INFORMATICS INSTITUTE OF TECHNOLOGY

In Collaboration with

UNIVERSITY OF WESTMINSTER

AN ADAPTIVE, CONFIDENCE-WEIGHTED VERIFICATION FRAMEWORK FOR MITIGATING LARGE LANGUAGE MODEL HALLUCINATIONS

Project Proposal and Requirements Specification by

Mr. Shaun Yogeshwaran

Supervised by

Mr. John Sriskandarajah

This Literature Review is submitted in partial fulfilment of the requirements for the BSc (Hons) Computer Science degree at the University of Westminster

November 2025

ABSTRACT

Large Language Model hallucination - generating information which is plausible but factually incorrect - limits the potential for large-scale deployment of LLMs (Lei et al., 2023). Although both inference-time verification strategies (logic-based verifiers; Retrieval-Augmented Generation (RAG)) have improved factual correctness at the expense of a single, inflexible verification threshold (Lei et al., 2023), there are limitations with these approaches. The application of one-size-fits-all verification thresholds results in suboptimal tradeoffs between over-flagging valid, nuanced responses due to overly restrictive thresholds and missing hallucinations due to lax thresholds (Lei et al., 2023). This problem area is defined as the need for a dynamic adaptation process that will allow for the evidence-retrieved confidence levels to adaptively determine the degree of strictness of the logical verifier (Lei et al., 2023).

To address this, the research introduces a new verification algorithm—Confidence-Weighted CONLI—within an optimized two-stage pipeline called AFLHR Lite. The framework integrates a logical verifier and retrieval module, dynamically tuning verifier parameters using confidence values derived during retrieval. Specifically, the RAG component retrieves evidence and computes a numerical confidence value representing evidence trustworthiness. This confidence value is then used to adjust the strictness of the Natural Language Inference (NLI) classifier that performs verification.

To empirically evaluate the effectiveness of the proposed methodology, a three-condition experimental evaluation will be implemented on the HaluEval dataset (Li et al., 2023) to evaluate the efficacy of the proposed methodological implementation. This evaluation will include comparisons among three conditions (C1: RAG-only; C2: RAG + Standard CONLI; C3: RAG + Confidence-Weighted CONLI). It is hypothesized that the initial results of this evaluation will show that the C3 pipeline has achieved a statistically significant improvement in the F1-score for detecting hallucinations. Additionally, it is expected that the C3 pipeline will achieve lower rates of false positives (over-flagged) than the C2 control and provide better latency trade-offs than the C2 control.

Subject Descriptors:

- Computing methodologies → Natural language processing → Language models → Hallucination mitigation
- Machine learning → Inference-time factuality verification → Retrieval-Augmented Generation (RAG) architectures
- Machine learning → Natural Language Inference (NLI) → Adaptive thresholding and confidence scoring

Keywords: LLM Hallucination; RAG; NLI; CONLI; Confidence-Weighted Verification; Adaptive Thresholding; Over-flagging.

ABSTRACT ..... ***ii***

LIST OF FIGURES, TABLES, AND ABBREVIATIONS ..... ***v***

List of figures ..... **v**

List of tables ..... **v**

Abbreviations ..... **vi**

CHAPTER 01: INTRODUCTION ..... ***vii***

1.1 Chapter overview ..... **vii**

1.2 Problem background ..... **vii**

1.3 Problem definition ..... **x**

1.4 Research motivation ..... **xi**

1.5 Existing work ..... **xii**

1.6 Research gap ..... **xiii**

1.7 Contribution to body of knowledge ..... **xiv**

1.7.1 Problem domain contribution xiv

1.7.2 Research domain contribution xv

1.8 Research challenges ..... **xvi**

1.9 Research questions ..... **xviii**

1.10 Research aim ..... **xviii**

1.11 Research objectives ..... **xviii**

1.12 Project scope ..... **xx**

1.12.1 In Scope xx

1.12.2 Out of scope (potentially deferred to future work) xxi

1.13 Hardware/software requirements ..... **xxi**

1.13.1 Hardware Requirements xxi

1.13.2 Software requirements xxii
1.14 Chapter summary xxii
CHAPTER 02: LITERATURE REVIEW xxiii
2.1 Chapter overview xxiii
2.2 The landscape of hallucination in LLMS: a foundational taxonomy xxiv
2.3 Foundational and advanced mitigation strategies xxvii
2.3.1 Foundational Rag: The Lewis Et Al. (2020) Paradigm xxviii
2.3.2 Advanced verification and its pitfalls: self-verification and the recursive self-correction fallacy xxix
2.3.3 The Emergence Of Logic-Centric Verification: Moving Beyond Self-Verification xxx
2.3.4 The Uniform Threshold Limitation: The Unresolved Core Problem xxxi
2.4 Logic-based verification frameworks xxxiii
2.4.1 Natural Language Inference (Nli) For Verification xxxiii
2.4.2 Prior adaptive verification attempts and broader ML theory xxxiv
2.5 Comparison with state-of-the-art xxxv
2.6 Systematic literature search and novelty gap xxxvii
2.7 Chapter summary xxxvii
CHAPTER 03: METHODOLOGY AND PROJECT MANAGEMENT xxxix
3.1 Chapter overview xxxix
3.2 Research methodology xxxix
3.3 Development methodology xlii
3.3.1 Requirement elicitation methodology xliii
3.3.2 Design methodology xliii
3.3.3 Programming paradigm xliii
3.3.4 Evaluation methodology xlvi
3.4 Project management methodology xlvi
3.4.1 Gantt chart xlvii
3.4.2 Deliverables xlvii
3.4.3 Resource requirements xlviii
3.4.4 Risk and mitigation (Expanded with Severity and Frequency) xlix
3.4.4 Chapter summary l
CHAPTER 04: SOFTWARE REQUIREMENT SPECIFICATION (SRS) li
4.1 Chapter overview li
4.2 Rich picture diagram li
4.3 Stakeholder analysis lii
4.4 Selection of Requirement Elicitation Methodologies lix
4.5 Discussion of findings lx
4.5.1 Literature review findings lx
4.6 Summary of Findings lxiii
4.6 Context diagram lxiv
4.7 Use case diagram lxiv

4.9 Use case descriptions ..... **lxv**
4.8 Requirements ..... **lxvi**
4.8.1 Functional Requirements (Fr) lxvi
4.8.2 Non-Functional Requirements lxviii
4.8.3 Chapter summary lxix
CHAPTER 05: TIME SCHEDULE ..... ***lxx***
5.1 Project schedule (Gantt chart) ..... **lxx**
References ..... **lxxi**

LIST OF FIGURES, TABLES, AND ABBREVIATIONS

List of figures

Figure 3.1: AFLHR Lite System Architecture
Figure 3.3: Confidence-Weighted CONLI Process Flow
Figure 4.1: Rich Picture Diagram
Figure 4.2: Context Diagram
Figure 4.3: Use Case Diagram

List of tables

Table 1.1: Existing Work Summary
Table 1.2: Research Objectives
Table 1.3: Hardware/Software Requirements

Table 2.1: Systematic Literature Search for Adaptive NLI Verification

Table 2.2: Comparative Analysis: SGIC (Chen et al., 2025) vs. AFLHR Lite (This Work)

Table 3.1: Saunders' Research Onion

Table 3.2: Algorithmic Pseudocode for Confidence-Weighted CONLI

Table 3.3: Experimental Design for Evaluation

Table 3.4: Risk and Mitigation Analysis

Table 4.1: Stakeholder Analysis

Table 4.2: Functional Requirements

Table 4.3: Non-Functional Requirements

Table 5.1: 4-Month Project Gantt Chart

Abbreviations

AFLHR: Adaptive Few-Shot Learning Framework for Hallucination Reduction

CONLI: Chain of Natural Language Inference

DPO: Direct Preference Optimization

LLM: Large Language Model

NLI: Natural Language Inference

RAG: Retrieval-Augmented Generation

SRS: Software Requirement Specification

CHAPTER 01: INTRODUCTION

1.1 Chapter overview

The purpose of this Chapter is to provide an overview of the research project described here — specifically to discuss the research problems and objectives that have led to the development of methods to reduce or eliminate hallucination in Large Language Models (LLMs) — and describe how the work described here has come to be defined. This Chapter first provides background information about the problem I am addressing, describing the rapid development of LLMs and the critical challenges they pose with regard to producing factually and logically sound content. The Chapter then formally defines the research problem being addressed, and narrows the research problem to the need for a higher verification threshold than those currently used to address the research problem. A concise problem statement is provided to facilitate further discussion of the research problem. The Chapter surveys related work to demonstrate the existence of a significant, currently unaddressed research gap. The Chapter identifies the major research questions and objectives that serve as the basis for the work described here. The Chapter then describes the specific scope of the project described here — a streamlined "AFLHR Lite" pipeline — and specifies the computer hardware and software required to complete the project successfully within a 4-month time frame. Finally, the Chapter explicitly addresses other components of the project as future work. The original Llama-3-8B (llama3-8b-8192) model was deprecated by Groq during development. It has been replaced with llama-3.1-8b-instant, which provides faster inference and improved response quality. All subsequent references to Llama-3 in this document refer to this upgraded version. Embedding Methodology: The sentence-transformers library was replaced with direct transformers library usage (AutoTokenizer + AutoModel with manual mean pooling) to avoid segmentation faults on Apple Silicon. The embedding output is functionally identical to standard SentenceTransformer behavior.

1.2 Problem background

The accelerated proliferation of Large Language Models (LLMs) throughout the world's most influential industries presents a groundbreaking shift in AI technology, capable of transforming

multiple sectors, including content creation, customer service, scientific research, and software development. As well as being able to process, generate, and manipulate human language with an unprecedented level of context-awareness and fluency, these models provide the means to automate processes, synthesize knowledge, and create interactive experiences that had previously not been possible.

One major barrier preventing them from becoming widely accepted for use in high-risk, mission-critical environments is their tendency to "hallucinate" — to produce seemingly plausible yet completely fabricated information. Hallucination represents a fundamental barrier to establishing reliable and trustworthy LLM-based systems.

Hallucination is a broad term used to describe the failure of LLMs to generate coherent, informative, or accurate responses to user input. It includes a wide variety of errors, including factual inaccuracies, logical inconsistencies within the generated response, extrinsic hallucinations (i.e., when the generated response contradicts the source material that was provided to the LLM), and intrinsic hallucinations (i.e., when the generated response contradicts itself).

These types of errors arise due to the probabilistic nature of LLMs, which are designed to predict the next item in a sequence. Because the design objective of LLMs is focused on producing fluent, coherent responses that may not necessarily be factually accurate, they can produce credible-sounding but entirely fabricated information. These types of errors pose significant risks to users, including the dissemination of misinformation, threats to safety, and limitations on the reliability of LLMs in critical domains like healthcare, law, and finance.

The ramifications of the hallucination problem are numerous and far-reaching. First, hallucinations reduce user faith in LLM systems, particularly in those applications where factual accuracy is paramount. In a medical diagnosis system or a legal document analysis system, a single hallucination could result in catastrophic consequences. Second, hallucinations create barriers to integrating LLMs into business systems and regulated industries where accountability and auditable documentation are non-negotiable requirements. Finally, hallucinations create additional computational and human effort, as domain experts must manually validate or correct LLM-generated outputs. Additionally, the manual validation of LLM-generated outputs can create significant barriers to efficiency because of the potential volume of LLM outputs that need to be

validated, as well as the cost and difficulty of validating the outputs in complex and highly-specialized domains.

To date, researchers have employed a variety of inference-time strategies to mitigate this hallucination problem. One strategy, Retrieval-Augmented Generation (RAG), developed by Lewis et al. (2020), incorporates retrieval of relevant information from an external database into the model's generation process to reduce reliance upon the model's potentially outdated or inaccurate internal knowledge. Although RAG establishes a robust baseline for improving the factuality of generated responses, it does not ensure that the model will accurately or logically use the context that is provided during generation.

More advanced logic-centered verification techniques, such as Chain of Natural Language Inference (CONLI), have been proposed to verify whether a generated statement (hypothesis) is logically implied by the original source text (premise) that is provided to the model. These verifiers employ NLI models trained on large-scale datasets to determine the relationship between premise-hypothesis pairs as either Entailment, Neutral, or Contradiction.

Despite their sophistication, however, all of the previously-proposed verification techniques have a critical flaw: they all employ fixed, uniform verification thresholds. For instance, a standard CONLI verifier would require that the NLI entailment score be greater than some fixed threshold value (e.g., 0.95) to consider a claim to be verified.

The "one-size-fits-all" approach of these previous verification techniques creates an undesirable tradeoff. On one hand, setting a threshold that is too stringent to detect the smallest, most hidden hallucinations will also lead to incorrect rejection of a large number of valid, nuanced, or abstractive claims that are not a direct, word-for-word entailment of the source text. This phenomenon is called "over-flagging." As a consequence, these previous verification techniques have resulted in extremely high rates of false positives, decreased usability, and longer processing times caused by the need for repeated processing of claims that are determined to be false. Conversely, setting a less restrictive threshold will result in the failure to detect actual hallucinations, thus undermining the effectiveness of the verifier and increasing the concern about the reliability of LLMs in mission-critical domains.

Because of this tradeoff between precision and recall, prior approaches to addressing this problem in the LLM hallucination domain have treated it as a necessary trade-off, rather than as an engineerable problem. The main reason for this lack of flexibility in verification thresholds is that current verification pipeline designs are "dumb," i.e., they are unaware of the quality of the evidence that they are using to support their verifications. Regardless of the relevance or trustworthiness of the documents supporting a claim, the downstream verifier applies a uniform pass/fail criterion to a claim that is supported by a 99% relevant document as well as to a claim that is supported by a 65% relevant document. Understanding this is important: the Retrieval-Augmented Generation component upstream of the verifier produces a confidence score for each of the retrieved documents (for example, via cosine similarity in vector databases) that serves as a numerical measure of evidence trustworthiness. Typically, however, this valuable information is wasted, and the downstream verifier cannot make use of it. The disconnection between these two components of the verification process represents a lost opportunity for developing a more flexible, more adaptive verification methodology that can dynamically adjust the stringency of the verifier based upon the trustworthiness of the evidence that it is verifying against.

1.3 Problem definition

The main issue that this research focuses on, is the current inflexibility and inefficiencies of today's real-time verification approaches used with LLMs (Lei et al., 2023). Although retrieval-augmented generation (RAG) provides a way to connect model output to external data, current verification techniques of whether this external data is being properly used has its own set of shortcomings (Lei et al., 2023). A number of logic-based verification techniques have been suggested including Chain of Natural Language Inference (CONLI) (Lei et al., 2023); however, current implementations of these verification tools have utilized a singular, uniform entailment threshold (e.g., nli_entailment_score > 0.95) (Lei et al., 2023).

Utilizing a single "one-size-fits-all" entailment threshold produces an essential, unsolved trade-off:

- A high (stringent) threshold (e.g., 0.99) is needed to identify fine-grained, hidden hallucinations; however, a high threshold will also result in an over-abundance of false

positive results, decreased system coverage and longer processing time due to the need for re-processing (Lei et al., 2023).

- A lower (lenient) threshold (e.g., 0.85) increases system speed and coverage, while at the same time misses true hallucinations and fails to accomplish the primary function of the verifier (Lei et al., 2023).

This fundamental problem is due to the fact that the verification layer is "oblivious" to the quality of evidence that it is utilizing (Lei et al., 2023). The verification layer uses the same strict pass/fail criterion for a claim based upon a 99% relevant document as it does for a claim based upon a 65% relevant document (Lei et al., 2023). This rigidity is another known impediment in other verification processes where "pass criteria may be too rigid, and therefore filter out good and accurate information" (Lei et al., 2023). This research identifies the core problem as the inability of the verification process to adjust to varying levels of relevance and quality of evidence (Lei et al., 2023).

1.3.1 Problem statement

The reliance on uniform-threshold verification mechanisms results in a rigid and suboptimal hallucination mitigation pipeline, necessitating an adaptive framework that can dynamically calibrate verification strictness based on the confidence of the retrieved evidence (Lei et al., 2023).

1.4 Research motivation

This research was prompted by the large technical difficulty associated with developing a verification tool capable of auditing an LLM's generated content (output) without limiting its ability to generate meaningful content, or causing undue delay in the generation process. The development of a verification system will need to maintain a fine balance between factually accurate output, logically consistent output, and fluent conversational output. A verification system that filters aggressively and/or rigidly may generate factually accurate responses, but

responses that are influent and therefore not useful; thus, reducing the value of the LLM. Therefore, the technical motivation of this research is to develop a "plug-and-play" verification algorithm that can be used at inference time to effectively address this trade-off. Through a focus on a computationally efficient two-layer "AFLHR Lite" pipeline, the objective of this research is to create a practical, available, and scalable solution for addressing this problem that is more economically viable than retraining the entire model.

1.5 Existing work

The field of hallucination mitigation is rapidly evolving. A foundational technique is Retrieval-Augmented Generation (RAG), which aims to ground LLM responses by first retrieving relevant information from an external knowledge base (Lewis et al., 2020). While RAG provides a strong baseline, it does not guarantee that the model will faithfully or logically utilize the provided context. Building upon this, self-verification techniques have emerged, with Chain-of-Verification (CoVe) being a prominent example (Dhuliawala et al., 2024). The following table summarizes key existing research relevant to this project.

Table 1.1: Existing Work Summary

Citation	Summary	Limitation	Contribution
Lewis et al. (2020)	Introduced Retrieval-Augmented Generation (RAG) to ground LLM outputs in external knowledge, improving factuality.	Does not inherently verify if the model faithfully or logically uses the retrieved context.	Established the paradigm of combining parametric and non-parametric knowledge for generation.

Dhuliawala et al. (2024)	Proposed Chain-of-Verification (CoVe), a self-verification method where an LLM generates a plan to check its own facts.	Fails to detect logical errors between facts; suffers from the "recursive self-correction fallacy".	Demonstrated the potential of self-correction but highlighted its core vulnerabilities.
Lei et al. (2023)	Introduced Chain of Natural Language Inference (CONLI) as a logic-centric method for hierarchical hallucination detection.	Introduces latency; standard implementations rely on rigid, uniform verification thresholds.	Provided a method for detecting hallucinations without external knowledge sources.
Manakul et al. (2023)	Introduced SelfCheckGPT, which uses sampling from the LLM to check for consistency and identify hallucinations.	Primarily detects inconsistencies within the model's own knowledge, not necessarily against ground truth or logic.	Provided a method for detecting hallucinations without external knowledge sources.

1.6 Research gap

A critical review of the literature reveals a significant and novel research gap. The field has two distinct, but currently un-bridged, areas of research:

1. Hallucination Verification: This includes work on post-hoc verifiers like CONLI (Lei et al.,

2023) and CoVe (Dhuliawala et al., 2024), which attempt to validate a generated output after it has been drafted.

2. Retrieval Calibration: This separate body of work, primarily in Information Retrieval (IR) and RAG optimization, explores the use of retrieval confidence scores to calibrate downstream tasks. Research has shown that confidence scores can "reliably dictate the trustworthiness of LLM outputs" (Tao et al., 2024) and that "uncertainty scores" can determine the "relevance of each document" (Chen et al., 2025).

The research gap is the absence of a bridge between these two fields. As detailed in the systematic literature review (Chapter 2, Table 2.1), no integrated framework was found that uses retrieval confidence as a weighting factor to dynamically modulate the parameters of a logical entailment (NLI) verifier. Therefore, to the best of my knowledge, this project is the first to propose and empirically test this specific mechanism.

Within a 4-month time and computational constraints, this project justifiably defers the complex, full-scale AFLHR system (which included cross-model checks and surgical correction). It pivots to focus on this most novel and high-impact component—the Confidence-Weighted CONLI algorithm—which serves as the foundational first step for any future multi-layer system.

1.7 Contribution to body of knowledge

This research is poised to make several significant contributions to both the problem domain of AI-generated content reliability and the research domain of AI safety and evaluation.

1.7.1 Problem domain contribution

This project's main contribution is developing the Confidence-Weighted CONLI (Cw-CONLI) algorithm; an entirely new way to apply retrieval confidence within the logical verification loop,

and thereby move past using the same rigid, static confidence threshold for all cases to the use of dynamic, context-specific thresholds for verification. All current NLI-verifier implementations use rigid, static thresholds to make the decision about whether a response is valid or invalid. However, these rigid thresholds create a trade-off: the rigid threshold can be very effective at catching hallucinations (false positives) but too rigid a threshold creates many false negatives by not accepting even nuanced responses. A less-rigid threshold can effectively validate responses but misses many subtle hallucinations. This is significant because the amount of verification that is done should depend on how good the retrieval evidence is; if the retrieval evidence is high-quality and therefore has high retrieval confidence, then it makes sense to be more strict when verifying the response; however, if the retrieval evidence is poor quality and therefore has low retrieval confidence, it is reasonable to be less strict when validating the response. Yet, to date, no verifier has been able to take advantage of this insight; they are all "blind" to the relevance of the retrieval confidence to validation. To address this, the Cw-CONLI algorithm uses cost-sensitive learning to map the retrieval score to dynamic thresholds, allowing the threshold to be dynamically modulated based upon retrieval confidence without requiring any additional training data. This is novel because previous research either used the same rigid threshold for all cases (Lei et al., 2023 CONLI) or calibrated the confidence of generated responses rather than validating them (Chen et al., 2025 SGIC) and did not systematically integrate retrieval confidence into threshold selection. The success of the Cw-CONLI algorithm will be evaluated using rigorously designed and executed controlled experiments (there will be three experimental conditions: RAG only, RAG + standard CONLI, and RAG + Cw-CONLI) using the HaluEval test set, and I expect the Cw-CONLI algorithm to result in a ≥ 5% improvement in F1-scores and a ≥ 10% reduction in false positives with ≤ 20% increase in latency overhead. After completing the experimentation, I will publicly release the Cw-CONLI algorithm, the experiment framework, and all reproducible artifacts as open source software so that other researchers can build upon our contributions, adopt the Cw-CONLI algorithm in their production RAG systems, and expand the scope of application to specialized domains.

1.7.2 Research domain contribution

The project will provide a very detailed, reproducible, three condition empirical study, measuring the accuracy-latency trade-off of applying dynamic versus uniform thresholding to 35,000+ hallucination examples of HaluEval; enabling the RAG development community to have specific, empirical data to support their decision-making in designing more effective verifiers for RAGs. In order to measure the performance increase due to confidence weighted thresholds, the study uses a controlled experimental approach where the performance difference between: (C1) the base case RAG-only responses, (C2) RAG responses using a global uniformly optimized threshold for CONLI, and (C3) RAG responses using dynamically weighted per inference thresholds for CONLI are compared using the metrics of: F1 score, precision, recall, number of false positives and wall-clock time. The study measures if context aware verification is better than one size fits all verification techniques, and at what computational expense. As well as the immediate performance gains (minimum of 5% higher F1 scores, minimum of 10% fewer false positives), the validation of the AFLHR Lite pipeline provides a fundamental basis for future research. The detection logic used in this phase of the system will be able to trigger the second phase of the system (the surgical correction layers that produce factually corrected responses) and also enable the third phase (the programmatic generation of high quality preference pairs for DPO knowledge distillation -- a method to incorporate reasoning into small models.) By publishing all of the source code, evaluation scripts and trained artifacts for the project as open source software, it provides the ability for reproducibility in the academic community, for practical use by companies developing production RAG systems and for systematic investigation of confidence-weighted verification in various application areas (e.g., medical, law, scientific literature).

1.8 Research challenges

This project addresses several significant technical and methodological challenges that must be carefully managed to ensure successful completion. These challenges, rooted in the complexity of both LLM verification and adaptive thresholding, are documented here with their corresponding mitigation strategies.

Optimal Threshold Tuning and Model Convergence

The central premise of this research project is that it is possible to improve performance with dynamically weighted NLI thresholds (C3), versus uniformly-tuned thresholds (C2). There is a

substantial risk, however, that even after extensive development set parameter tuning, the dynamic model (C3) will be unable to demonstrate significant improvement over the control (C2) baseline. A number of factors can contribute to this failure including: If there is little relationship between retrieval confidence scores and optimal NLI thresholds; the relationship is complex and/or non-linear; the relationship is noisy; or if the tuning process converges to local optima instead of global optima. In addition, the large number of parameters that are tunable to determine the optimal NLI thresholds (RETRIEVAL_LOW_CONF, RETRIEVAL_MOD_CONF, RETRIEVAL_HIGH_CONF, T_STRICT, T_LENIENT) can result in overfitting to the development data with subsequent poor generalization to the blind test data.

Noisy and Poorly-Calibrated Retrieval Scores (Medium Severity, High Frequency)

The AFLHR Lite algorithm fundamentally depends on the quality and calibration of the retrieval confidence scores produced by the RAG system's vector database (FAISS). These scores (e.g., cosine similarity in semantic space) are proxy measures of document relevance and are not guaranteed to be well-calibrated; they may be biased toward high values, clustered in a narrow range, or poorly correlated with actual document quality. For example, if the FAISS index produces similarity scores that are uniformly high (e.g., 0.85- 0.95) across diverse documents of varying quality, then the signal-to-noise ratio for the dynamic threshold adjustment is diminished. Conversely, if scores are uniformly low or exhibit a non-linear relationship with document quality, the tiered-threshold logic may fail to capture the true gradient of evidence quality. This noise would directly undermine the premise of the confidence-weighting mechanism.

Computational Resource Constraints and GPU Memory Management

The project's computational budget is constrained. The AFLHR Lite pipeline requires simultaneous loading of three large models: (1) the Llama-3-8B-Instruct generative model (~16GB in bf16 precision), (2) the roberta-large-mnli NLI classifier (~1.4GB), and (3) FAISS indices and runtime buffers (~3GB). This totals approximately 20GB of VRAM, pushing the limits of commonly available GPU resources (Google Colab Pro V100: 16GB, A100: 40GB, T4: 16GB). If the available GPU has less than 20GB of VRAM, the pipeline will fail due to out-of-memory (OOM) errors.

1.9 Research questions

This research is guided by the following primary questions:

- RQ1: To what extent does dynamically weighting Natural Language Inference (NLI) verification by retrieval confidence improve hallucination detection accuracy (F1-score) compared to a uniform-threshold approach?
- RQ2: How does the Confidence-Weighted CONLI pipeline affect key system-level performance metrics, specifically query latency and the rate of "false positives" (over-flagging)?
- RQ3: Which tuning strategy—the tiered-threshold model (as defined in Table 3.2) or an alternative continuous-weighting function (e.g., sqrt-based)—yields a superior F1-score on the HaluEval dev set, and what are the specific hyperparameters that maximize this performance?

1.10 Research aim

The aim of this research is to design, implement, and rigorously evaluate a two-layer hallucination mitigation pipeline (AFLHR Lite), centered on a novel Confidence-Weighted CONLI algorithm, to empirically measure its improvements in detection accuracy and latency trade-offs compared to uniform-threshold approaches.

1.11 Research objectives

To achieve the stated aim, the following specific objectives have been defined. The metrics included in the descriptions for RO2, RO3, and RO4 represent the specific, measurable success criteria for the project. These metrics are interconnected: achieving the F1-score gain (RO3) while simultaneously reducing false positives (RO4) is the key indicator that the adaptive mechanism is successfully solving the "over-flagging" trade-off, and doing so within the latency budget (RO2) proves its real-world viability.

Table 1.2: Research Objectives

ID	Research Objective	Description	RQ Mapped	LOs Mapped
RO1	To design a confidence-weighting algorithm.	Design an algorithm that dynamically adjusts NLI verification thresholds based on RAG retrieval confidence scores.	RQ1	LO1, LO3
RO2	To implement the AFLHR Lite two-layer pipeline.	Implement the algorithm by integrating a RAG component with a CONLI verifier. Success criterion: Latency increase vs. the C2 (Standard-CONLI) baseline.	RQ1, RQ2	LO2, LO3
RO3	To conduct a rigorous empirical evaluation.	Design and execute a 3-condition experiment (Baseline vs. Standard-CONLI vs. Weighted-CONLI) on a human-annotated dev set from the HaluEval benchmark (Li et al., 2023). Success criterion: Achieve F1 improvement over C2.	RQ1, RQ2, RQ3	LO3, LO4

RO4	To analyze empirical trade-offs.	Analyze the resulting data to quantify the trade-offs between hallucination detection (F1-score), latency, and over-flagging. Success criterion: Reduce false positives (over-flagging) vs. C2.	RQ2, RO3	LO4

1.12 Project scope

1.12.1 In Scope

- Core Framework: The design, implementation, and evaluation of the AFLHR Lite pipeline will be orchestrated using the LangChain library framework.
- RAG Pipeline Components: The foundational RAG layer will be constructed with the following components:
  - Knowledge Base: A curated corpus from the English Wikipedia.
  - Embedding Model: sentence-transformers/all-MiniLM-L6-v2.
  - Vector Store: An in-memory FAISS index (Johnson et al., 2019).
  - Retrieval Score Extraction: The pipeline will be configured to use the vector store's similarity_search_with_relevance_scores method, or equivalent, to extract both the source documents and their numerical confidence scores (Douze et al., 2025). This score is a critical output.
- Core Generative Model: The pipeline will use meta-llama/llama-3.1-8b-instant as the generator LLM for all experimental conditions.
- Verification Component: The programmatic Confidence-Weighted CONLI layer will be implemented. This includes:

  - Using a pretrained, lightweight NLI classifier, FacebookAI/roberta-large-mnli (Liu et al., 2019; Williams et al., 2018), to generate objective entailment, neutral, and contradiction scores.
  - Implementing the dynamic thresholding algorithm (RO1).
- Evaluation Harness: Development of a testing harness to automatically run the 3-condition experiment (RO3) on the HaluEval benchmark (Li et al., 2023) and compute all key metrics (RO4).

1.12.2 Out of scope (potentially deferred to future work)

The following components, part of the original, larger AFLHR vision, are explicitly de-scoped from this 4-month project to ensure focus and feasibility.

- Cross-Model Consistency Layer: The use of a secondary, architecturally distinct model (e.g., Mistral-7B) for verification is deferred. This is a powerful but computationally expensive technique that represents a separate, subsequent research phase.
- Surgical Correction Layer: The development of a post-processing layer to correct detected hallucinations is out of scope. This project focuses on perfecting detection, which is the necessary prerequisite for any correction.
- DPO Knowledge Distillation (Stretch Goal): The stretch goal to "internalize" the pipeline's behavior using Direct Preference Optimization (DPO) is deferred and repositioned as a Phase 3 future work. This deferral is methodologically sound: this project must first build and validate the "teacher" (the AFLHR Lite engine) before its logic can be "distilled" into a "student" model.

1.13 Hardware/software requirements

1.13.1 Hardware Requirements

Component	Specification

Processor (CPU)	Apple M2, 8-core
Memory (RAM)	8 GB Unified Memory
Storage	256 GB SSD
GPU / Compute	Cloud-based GPU (Google Colab / AWS EC2 / OpenAI API) VRAM of 20GB+

1.13.2 Software requirements

Component	Specification
Python and libraries	Version 3.10 or higher
LangChain	Latest stable release
Google Colab	Paid version
MS Word / Google Docs	Paid version
FAISS	Latest stable
VS Code	Latest version
Git & GitHub	Latest version

1.14 Chapter summary

This chapter has established the critical problem of LLM hallucinations and highlighted the shortcomings of existing mitigation solutions, specifically their reliance on rigid, uniform verification thresholds (Lei et al., 2023). It has defined a clear and novel research gap—the lack of integration between retrieval confidence and logical verification—and proposed the "AFLHR Lite" pipeline, centered on a "Confidence-Weighted CONLI" algorithm, to address this gap. The research aim, guiding questions, and specific, measurable objectives have been clearly outlined. Furthermore, the project scope has been precisely defined to a feasible 4-month timeline, prioritizing the core algorithmic and empirical contributions while strategically deferring more

complex layers as future work.

CHAPTER 02: LITERATURE REVIEW

2.1 Chapter overview

This chapter provides a complete overview of the literature that currently exists regarding the detection and reduction of hallucinations in LLMs. This chapter will start off by establishing a formal classification system for the different types of hallucinations which is needed to place the issue into an academic context. Next, the chapter will discuss the various methods of detecting and reducing hallucinations in LLMs, and how they have evolved over time, specifically RAG and how it has developed into a developing area of study. Following this, the chapter will establish a base level of knowledge concerning NLI to support the methodologies used to verify the results of this project, through a review of the NLI datasets and models available.

In addition to presenting the current state of knowledge regarding the development of algorithms designed to detect and reduce hallucinations in LLMs, this chapter also presents a significant amount of additional analysis. Specifically, it explores the reason for the persistence of the "uniform threshold" problem; in terms of the fundamental trade-offs of accuracy and complexity that are present when developing any algorithm. Additionally, it reviews prior work on adaptive thresholding in the field of machine learning (such as "threshold moving" and "cost sensitive learning") and establishes a solid theory-based approach to develop the algorithm presented in this project. A detailed comparative analysis of this proposed algorithm and the current state of the art, SGIC (Chen et al., 2025), was conducted to clearly define the contributions of this project. Lastly,

the results of a systematic literature search are presented, showing the existence of the novel, un-bridged research gap addressed by this proposal.

2.2 The landscape of hallucination in LLMS: a foundational taxonomy

Hallucination in LLMs is the generation of fluent, syntactically correct content that is factually inaccurate or ungrounded in any provided source material. This phenomenon undermines the reliability of LLMs and is a major obstacle to their deployment in high-stakes domains.

Research has categorized hallucinations into two primary types. This foundational dichotomy was introduced by Maynez et al. (2020) in a seminal analysis of abstractive summarization.

- Intrinsic Hallucination: The generated output directly contradicts the source content. This is a failure of logical faithfulness.
- Extrinsic Hallucination: The generated output contains information that cannot be verified from the source material. This information may be factually correct in the real world, but it is ungrounded in the provided context.

Hallucinations by large language models (LLMs) represent an emerging, highly important field of research within both AI Safety and NLU as LLMs achieve greater scale and capabilities than ever before, are capable of creating, with varying degrees of success, human-like text for a wide variety of areas including creative writing, technical writing, and medical diagnostic support. An equally significant challenge emerges simultaneously, as LLMs create fluent, grammatically correct, and semantically accurate text which is verifiably inaccurate in terms of factuality or logical consistency. Hallucinations thus constitute a major obstacle to the deployment of LLMs in high-stakes/mission critical environments, where factuality is non-negotiable.

The problem domain of LLM hallucination mitigation is fundamentally multi-disciplinary; it involves consideration from many points-of-view. From a purely technical standpoint, hallucinations arise due to how Large Language Models are designed: as probabilistic systems which predict the next token in a sequence given previously learned statistical patterns from the training data. The objective of designing a model to maximize fluency and coherence can diverge significantly from maximizing factual accuracy. For example, a model that maximizes fluency and coherence of continuations may produce fictional information if there is limited information

available for obscure topics, outdated information if the training data is out-dated, or logically incoherent responses when the task requires reasoning that exceeds the capabilities of the model.

Hallucination's degree of harm from an application perspective has the potential to vary greatly depending on the domain. Hallucination's degree of harm can be minimal in many areas of applications; including, however, entertainment, creative writing, brainstorming, etc., it is possible that small amounts of inaccurate information may be permissible or even preferable. In contrast, in many other areas where the consequences of hallucination can have major negative impacts, including health care (where a LLM generated misdiagnosis could pose serious risk to a patient's safety); legal services (where manufactured case citation references could potentially expose attorneys to liability for malpractice); finance (where a LLM generated, inaccurate market analysis could result in millions of dollars in loss); and research (where manufactured references could damage the integrity of the academic record) hallucination would be considered a critical failure mode. This domain dependent severity of hallucination results in hallucination mitigation being perceived as not a single problem but rather a series of distinct, domain specific problems that will require unique calibration of the accuracy constraint trade offs against the level of utility allowed by the system.

Hallucination has several significant ramifications in addition to producing individual errors. Even after hallucinated content has been identified as error and has been corrected, there is a substantial amount of damage done to user confidence in the system. Companies using LLMs in highly regulated environments are also under increased scrutiny: regulatory agencies are now demanding documentation of how reliable a system is; the ability to audit the decision making process of a system; and documentation of responsibility for system errors. Hallucinations that remain uncorrected create liability issues and prevent companies from implementing these systems in their regulated business environments. Finally, the human resources needed to verify the outputs produced by LLMs can be significant in cost and time, especially when verifying specialized knowledge requires review by experts. Therefore, in batch processing environments, if a LLM generates output with a hallucination rate of just 5-10%, the output will likely need to be verified by humans and therefore the use of LLM based systems in this environment will be economically infeasible.

This challenge has been met with several strategies from the technical research community, with each strategy focusing on a different aspect of the hallucination problem. As a major strategy, Retrieval-Augmented Generation (RAG) is a leading solution that addresses the central issue that LLMs do not have up-to-date knowledge in real time and therefore can only rely on out-of-date or incomplete internal knowledge representations. The integration of external document retrieval into the generation process enables RAG to base model generated responses on current, verifiable sources. Nonetheless, it is possible that the information retrieved by a model could be accurately reflected in the generated response, while the model itself still misinterprets, misquotes, or misuses the retrieved information in the generated response. The potential to allow models to generate responses based on inaccurate interpretations of accurate information has led to the development of verification methods to determine if generated claims are logically consistent with their source materials.

Verification methods using logic, including Natural Language Inference (NLI), aim to formalize the concept of "faithful representation" of the generated claim. Verification methods use logic to decompose the generated claim into individual propositions and verify whether each proposition is logically entailed by the source material (premise). These verification methods provide a model-independent, objective mechanism for detecting hallucinations. However, verification methods also present a new methodological challenge of determining the optimal verification threshold. A verification threshold that is set too high will result in rejecting valid, abstracted or nuanced claims that accurately represent the source materials, although the claim may not literally reproduce the words of the source materials. Conversely, a verification threshold that is set too low will result in failing to detect subtle hallucinations or claims that inaccurately represent the source materials. As such, the trade-off between precision (avoiding unnecessary rejection of valid claims by avoiding false positives) and recall (avoiding failure to detect actual hallucinations by avoiding false negatives) continues to remain an open question within the research field.

This tradeoff is not coincidental; rather, it is an indication of the fundamental engineering problem at play. In its current form, the verification mechanism operates blindly with regard to the quality of evidence supporting a particular claim. Once a verifier has received a claim to assess, it uniformly applies a decision criterion to all such claims regardless of the reliability of the evidence on which they are based. As such, the upstream Retrieval Aggregation (RAG) component

generates a valuable signal -- a retrieval confidence score representing the degree to which each retrieved document matches the original query -- and provides a quantifiable measure of the quality of the evidence used to support the claim. However, the RAG confidence scores are generally discarded by the downstream verification layer, thereby creating an opportunity to create a more sophisticated, dynamic verification process that can adaptively vary its level of scrutiny as a function of the quality of the evidence supporting each claim.

Thus, the research gap addressed by this dissertation is the connection between these two otherwise disconnected components of the overall verification process. Both Hallucination Verification and Retrieval Calibration represent active areas of research, however the integration of the upstream RAG confidence signals into the dynamic, adaptive verification thresholds has not been explored in detail. The underlying assumption driving this research is a simple one, yet profound in its implications. That is, the level of scrutiny applied during the verification process should be directly related to the quality of evidence supporting each claim. When high-quality, highly-relevant evidence exists, the verification process should apply a higher level of scrutiny in order to minimize the cost associated with false positive errors (i.e., incorrectly rejecting valid claims), while minimizing the cost of false negative errors (i.e., missing rare hallucinations). Conversely, when poor-quality or marginally relevant evidence exists, the verification process should be less stringent in terms of requiring exact logical entailment, but proportionately stricter with regards to identifying contradictions or logical inconsistencies. Such adaptive calibration would significantly enhance the practical application of verification-based hallucination mitigation systems.

The problem domain of LLM hallucination mitigation is thus characterized by multiple intersecting challenges: the inherent probabilistic nature of language models, the domain-dependent severity of hallucination impacts, the necessity of maintaining system utility while improving accuracy, the tension between precision and recall in verification, and the underutilization of retrieval confidence signals in verification pipelines. This research addresses a specific, well-defined subset of this broader problem domain: the development and empirical validation of a methodology for dynamically adjusting NLI-based verification thresholds using retrieval confidence scores within a computationally efficient, two-stage pipeline architecture.

2.3 Foundational and advanced mitigation strategies

2.3.1 Foundational Rag: The Lewis Et Al. (2020) Paradigm

Retrieval-Augmented Generation (RAG) has become one of the leading strategies to address the root cause of hallucinations in LLMs: their over-reliance on outdated and/or incomplete internal parametric knowledge. Developed by Lewis et al. (2020) RAG utilizes the straightforward yet highly effective strategy of retrieving relevant documents prior to generating responses, thus linking the model's output directly to verifiable, current information.

The RAG architecture follows a clear workflow. Initially, the user input question is converted to a compact vector representation via a pre-trained embedding model. The resulting vector is then matched against a vectorized database of document segments (most often through similarity metrics, i.e., cosine similarity). Once the most semantically similar documents have been identified they are then concatenated and fed into the generative LLM as additional context. The model then utilizes the provided contextual information to generate its response (hypothesis); it is anticipated that this contextualization would lead to reduced hallucinatory behavior as the model would be constrained to factual evidence in its generation of responses.

While RAG has demonstrated the ability to significantly reduce many types of hallucinations through the connection of LLM responses to external knowledge bases; specifically, it has been shown to effectively prevent unfounded claims regarding very recent events, domain-specific facts, and facts outside of the training data of the LLM. Thus, this provides a primary means of preventing hallucination.

Although foundational RAG has proven itself to be a crucial first line of defense against hallucination, it also has numerous shortcomings. First, there is no guarantee that an LLM will appropriately utilize the retrieved context information. For example, an LLM may intentionally distort, paraphrase, or select portions of the retrieved information in a manner that contradicts the original content. Second, the success of RAG is contingent upon the retrieval performance of the retrieval module; if the retrieval module is unable to identify relevant documents, the generative module has no basis of operation, and hallucinations may continue to occur. Third, and most importantly to this study, RAG generates a useful by-product that is typically ignored: the retrieval

confidence value for each retrieved document (i.e., the cosine similarity value between 0.0 and 1.0).

This retrieval confidence value represents a quantifiable indicator of how closely each retrieved document aligns with the user's query; therefore, it can serve as a numeric indicator of the validity/trustworthiness and relevance of the supporting evidence. Although this value holds significant potential for improving the performance of downstream systems (such as verification systems) that must assess the validity of claims produced by LLMs, typical RAG architectures discard the confidence values for the retrieved documents along with the retrieved text, essentially discarding potentially actionable information about the reliability of the supporting evidence at no computational cost. This discarded signal becomes especially meaningful when downstream modules require making critical decisions about accepting/rejecting claims generated by LLMs. It is logical to assume that knowing which claims are supported by strong confidence evidence (i.e., 0.95 similarity) versus weak confidence evidence (i.e., 0.60 similarity) could influence the degree to which a verification system evaluates those claims.

2.3.2 Advanced verification and its pitfalls: self-verification and the recursive self-correction fallacy

Given that a model's ability to retrieve information doesn't ensure it will apply the appropriate contextual information appropriately, researchers created verification techniques to determine if a generated response by an LLM actually applies contextually to the source data. Verification techniques were further categorized into three subgroups including the self-verification subgroup which involves a verification technique that prompts the model to create a verification method through the application of structured reasoning to provide evidence to support the validity of the model's responses.

A prominent example of a self-verification technique was Chain-of-Verification (CoVe) as described by Dhuliawala et al. (2024). CoVe creates a verification pipeline composed of four stages: stage one; the model provides a generated response, stage two; the model develops verification questions that would check whether specific claims made in the generated response are correct, stage three; the model responds to the verification questions, and stage four; the model develops a verified response based on the results of the verification questions. The rationale behind

CoVe is to prompt the model to generate verification questions that will allow the model to think through the verification of its own generated response. Therefore, the model may potentially self-correct its responses based on the verification process prior to delivering the response to the end-user.

Although self-verification methods have some theoretical merit in their potential to improve the accuracy of models, all self-verification techniques share a common weakness known as the "recursive self-correction fallacy." The recursive self-correction fallacy is defined as the tendency of a model trained using deep learning to reinforce its own incorrect assumptions because of a lack of intrinsic motivation or capability to refute prior statements that have been previously accepted as true. In addition, self-verification techniques typically only evaluate a model's responses at the propositional level, i.e. evaluating individual statements rather than evaluating the overall structure of a response or how various pieces of information are integrated into a coherent response.

In addition to self-verification, several other verification techniques have been developed, e.g. SelfCheckGPT (Manakul et al., 2023). The primary distinction between these other techniques and self-verification techniques is that other techniques involve sampling-based evaluation techniques that check the internal consistency of multiple samples produced by a model. Although these sampling-based techniques can detect internal inconsistencies within the knowledge base of a model, they do not verify generated claims against ground truth nor verify the generated claim based upon the logical relationships between the premises used to generate the claim and the claim itself.

Therefore, although the research conducted on self-verification has provided important insights into the importance of developing techniques to verify the outputs of LLMs and the inability of verification techniques to serve as objective arbiters of generated claims (i.e. verification techniques inherit the same hallucinations and circular reasoning exhibited by the original generative model), it also motivated the development of objective verification techniques that are independent of the LLM's limitations.

2.3.3 The Emergence Of Logic-Centric Verification: Moving Beyond Self-Verification

Researchers were motivated to develop new verification methods using formal logic and linguistic

theories since they realized self-verification could have many limitations. As a result, researchers made a paradigmatic shift away from internally verifying their own self-consistency to externally validating the logical consistency of a set of hypotheses about a given premise.

Verification using formal logic decomposes a very difficult problem—determining whether a particular, detailed hypothesis (claim) is logically entailed by a larger body of evidence (premise)—into a much simpler problem. This problem decomposition enables researchers to frame the problem of identifying hallucinations using the well-understood task of natural language inference (NLI), also called recognizing textual entailment (RTE). Researchers can treat the verification problem as a classification task with a clear semantic definition: for every premise-hypothesis pair, classify the relationship as either entailment (the hypothesis is logically entailed by the premise), contradiction (the hypothesis is logically contradictory to the premise), or neutral (there is neither a logically entailing nor contradictory relationship).

Using specialized NLI classifiers — especially those trained on large, diverse inference datasets, such as multi-nli (Williams et al., 2018) — has another important advantage. These classifiers provide an independent, objective measure of logical consistency. Importantly, unlike llms, which are designed to optimize fluency and coherence at potentially the cost of factuality, NLI classifiers are specifically designed to perform a classification task based on explicit ground truth labels.

Lei et al. (2023) developed a method called Chain of Natural Language Inference (CONLI) that uses this approach. Specifically, CONLI breaks down a generated response into individual sentences, treats each sentence as a hypothesis to be verified relative to the extracted premise. The NLI classifier produces a probability distribution across the three possible entailment relationships (entailment, contradiction, neutral). The system then extracts the entailment score (with scores ranging from 0.0 to 1.0) as the final decision signal. A key benefit of CONLI over self-verification is that it introduces an external, objective mechanism to verify the accuracy of a hypothesis about a premise, thus avoiding the recursive self-correction fallacy.

2.3.4 The Uniform Threshold Limitation: The Unresolved Core Problem

Though current verifiers that use a logic-based approach for verifying NLI models may appear

sophisticated, all such verifiers currently have a major drawback - a static verification threshold. The process for determining whether an NLI model has successfully validated a claim is as follows. First, the NLI model generates an entailment score for the claim generated. Then, using a rule based on the entailment score, if the entailment score is above a predefined, static threshold value (for example, 0.95), the claim is determined to be verified; otherwise, it is considered to be a hallucination.

A static verification threshold offers the benefit of simplifying the verification process from a computational standpoint, though it introduces a well-known and recognized trade-off with respect to the quality of the verification results. Setting the threshold sufficiently high to detect a large number of subtle and/or context-dependent hallucinations (for example, 0.99), a large number of accurate, nuanced, or abstractly phrased claims that correspond to the original content, but do not represent word-for-word entailment, will also be incorrectly classified as hallucinations. Over-flagging occurs when users are presented with valid claims that were incorrectly classified as hallucinations. Users lose trust in the system, which decreases the real-world usefulness of the LLM, even though the LLM was correct in its classification of the claim.

On the other hand, lowering the threshold to allow for greater system coverage and fewer instances of over-flagging (for example, 0.85) will increase system coverage and reduce over-flagging, though at the expense of failing to identify many cases of actual hallucinations. Low thresholds become overly lenient, causing systems to fail to detect subtle false statements and logical inconsistencies, thereby failing to perform one of the most important functions of the verification system.

The continued existence of this trade-off among all of the existing systems indicates that this is a deliberate engineering choice that has been made due to the well-known trade-off in machine learning, namely that increasing the complexity of a model to better solve a problem is typically offset by decreasing the practical feasibility of implementing the solution. As static thresholds are easy to implement, debug, and deploy, they add no additional computational overhead and are interpretable by design. For those applications in which occasional hallucinations are tolerated, the relative simplicity of static thresholds has proven to be sufficient. However, for applications in

which the potential consequences of a single hallucination are severe (healthcare, law, finance), the failure rate inherent in static thresholding is unacceptable, however the field has viewed this as an insurmountable trade-off rather than an engineerable problem.

2.4 Logic-based verification frameworks

2.4.1 Natural Language Inference (Nli) For Verification

To overcome the limitations of fact-centric verification, logic-based alternatives have been proposed. Natural Language Inference (NLI), or Recognizing Textual Entailment (RTE), is a fundamental Natural Language Understanding task that involves classifying the relationship between a pair of sentences (a premise and a hypothesis) as one of three labels: Entailment, Contradiction, or Neutral (Perikos et al., 2023). Note: The publication date for Perikos et al. has been corrected to 2023 from 2025, based on a verification of journal records.

The application of NLI to detect factual inconsistency and hallucination is a well-established field. Studies have shown NLI-based models to be effective at evaluating factual consistency in summarization (Kryscinski et al., 2020) and have re-visited NLI as a robust method for inconsistency detection (Laban et al., 2022). Frameworks such as Chain of Natural Language Inference (CONLI) (Lei et al., 2023) reframe verification as an NLI task. The generated response is broken down into individual sentences (hypotheses), which are then compared against the retrieved source text (premise).

This project utilizes a specialized classifier model to determine the logical relationship. The selection of the FacebookAI/roberta-large-mnli model for this task is a deliberate methodological choice. This model is a specialized, objective classifier, not a generative LLM. It is an

implementation of RoBERTa (Liu et al., 2019) that was fine-tuned on the Multi-Genre Natural Language Inference (MultiNLI) corpus (Williams et al., 2018). The MultiNLI corpus is a large-scale, 433,000-pair benchmark dataset covering 10 distinct genres of spoken and written text (Williams et al., 2018). Because models trained on this corpus (like roberta-large-mnli) are highly robust at performing cross-genre semantic understanding, it makes this model a reliable, non-hallucinating arbiter for the verification layer, superior to using a general-purpose (and self-biased) LLM for the same task.

2.4.2 Prior adaptive verification attempts and broader ML theory

The proposed Confidence-Weighted CONLI algorithm, while novel in its application, is built upon established principles in machine learning for handling classification under uncertainty, particularly with imbalanced data. The problem of "over-flagging" valid but-nuanced responses is analogous to a classic imbalanced data problem, where the "minority class" (valid, nuanced entailment) is frequently misclassified by a classifier biased toward the "majority class" (literal, obvious entailment).

The machine learning field has two primary approaches to this, which provide a theoretical foundation for this project:

1. Threshold-Moving: This is a common technique used in imbalanced classification. Instead of using a default 0.5 threshold, the decision threshold is "moved" (e.g., to a lower value) to increase the detection rate (recall) of the minority class. This new threshold is typically optimized on a validation set to maximize a metric like the F1-score, which balances precision and recall. The C2 (RAG + Standard CONLI) baseline in this project, which will be tuned on a dev set, represents this simple threshold-moving approach.
2. Cost-Sensitive Learning: A more advanced approach is to treat misclassification errors as having unequal costs. For example, a False Negative (missing a disease) is far more "costly" than a False Positive. Algorithms like MetaCost (Domingos, 1999) "wrap" a classifier, relabeling the training data to reflect these asymmetric costs, thus forcing the model to avoid high-cost errors.

The Confidence-Weighted CONLI algorithm can be understood as a novel, inference-time implementation of cost-sensitive learning. It uses the retrieval_score as a dynamic proxy for the misclassification cost matrix on a per-inference basis:

- When retrieval_score is LOW (e.g., 0.60): The system has low trust in the evidence.
  - The cost of a False Negative (accepting a hallucination) is considered extremely high.
  - The cost of a False Positive (rejecting a valid claim) is considered low (a "safe" error).
  - The algorithm, therefore, sets a strict threshold (e.g., 0.99) to avoid the high-cost error.
- When retrieval_score is HIGH (e.g., 0.90): The system has high trust in the evidence.
  - The cost of a False Negative is considered low (the evidence is good, the claim is likely true).
  - The cost of a False Positive (needlessly "over-flagging" a nuanced, valid claim) is now considered high, as it degrades the user experience.
  - The algorithm, therefore, sets a lenient threshold (e.g., 0.85) to avoid this new high-cost error.

This positions the proposed solution not as an ad hoc heuristic, but as a principled application of cost-sensitive learning theory to the domain of LLM verification.

2.5 Comparison with state-of-the-art

The field of retrieval calibration is evolving rapidly. A prominent new, state-of-the-art framework is SGIC (Self-Guided Iterative Calibration), proposed by Chen et al. (2025). A deep analysis of this work is necessary to situate this project's contribution.

- Reference Verification: The work by Chen et al. (2025), "SGIC: A Self-Guided Iterative Calibration Framework for RAG," is a published conference paper in the Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025), not a preprint.
- SGIC Methodology: SGIC is a framework to "enhance retrieval-augmented generation" by improving the LLM's "calibration capabilities" (Chen et al., 2025). It uses "uncertainty scores" to determine the relevance of documents and the confidence in the response (Chen et al., 2025). Its core mechanism is a self-guided iterative feedback loop: the model "reevaluates these scores iteratively, amalgamating them with prior responses to refine calibration" (Chen

et al., 2025). In short, SGIC is an advanced, iterative method to guide the retrieval and generation process itself, aiming to produce a better initial answer.

- Core Distinction (SGIC vs. AFLHR Lite): The SGIC framework and this project's AFLHR Lite are highly complementary and non-competing, as they operate on different stages of the pipeline.
    - SGIC calibrates the Retrieval and Generation (Layer 1) process. Its goal is to improve the quality of the hypothesis before it is finalized.
    - AFLHR Lite calibrates the Verification (Layer 2) process. It is a post-hoc check that assumes the hypothesis is already generated. Its goal is to make a better final judgment on that hypothesis.

An ideal, end-to-end factuality pipeline could (and perhaps should) use both: SGIC would be used in Layer 1 to produce the highest-quality, best-calibrated response, and AFLHR Lite would be used in Layer 2 as a final, cost-sensitive verification pass to ensure that response is logically entailed by the (high-quality) evidence. The relationship is symbiotic, not redundant. This distinction is summarized in the table below.

Dimension	SGIC (Chen et al., 2025)	AFLHR Lite (This Work)
Primary Goal	Calibrate RAG for improved response generation.	Calibrate NLI for improved response verification.
Point of Intervention	Pre-generation & Generation-time (Iterative) (Chen et al., 2025).	Post-generation (One-shot verification).
Calibration Target	The retriever (what docs to trust) and the generator (what words to write) (Chen et al., 2025).	The verifier (what NLI threshold to apply).

Methodology	Iterative self-guided feedback loop using uncertainty scores (Chen et al., 2025).	Dynamic, per-inference threshold modulation of a separate NLI model.
Relationship	Solves "How to write the best response?"	Solves "How to best judge the written response?"

2.6 Systematic literature search and novelty gap

To validate the novelty claim made in Section 1.6, a systematic literature search was conducted (November 2025) across Google Scholar, arXiv, and the ACL Anthology. The search aimed to find any existing work that "bridges the gap" between retrieval confidence and NLI verification.

The search methodology used a combination of keywords from the two distinct fields:

- NLI Verification Terms: "NLI", "textual entailment", "CONLI", "logic verification", "factuality", "uniform threshold"
- Retrieval Calibration Terms: "retrieval confidence", "retrieval score", "RAG confidence", "adaptive", "dynamic threshold", "confidence-weighted"

The findings, summarized in Table 2.1, confirm a clear and un-bridged research gap. While literature is plentiful on each side of the gap (NLI for verification; retrieval confidence for RAG calibration), no published work was found that implements the central mechanism of this proposal: using a RAG retrieval_score to dynamically modulate the entailment threshold of a post-hoc NLI verifier on a per-inference basis.

2.7 Chapter summary

This chapter has reviewed the literature on LLM hallucinations, establishing a formal taxonomy based on faithfulness and context inconsistency (Maynez et al., 2020; Ji et al., 2024). It moved beyond foundational techniques like "Naive RAG" (Lewis et al., 2020) to show that the retrieval

field is already evolving. This evolution was contrasted with logic-centric verifiers like CONLI (Lei et al., 2023) and other NLI-based methods (Laban et al., 2022; Wang et al., 2025), which, despite their power, suffer from a critical limitation: their reliance on rigid, uniform thresholds.

This critical analysis has established why this limitation persists, identifying the rational "simplicity vs. accuracy" trade-off that developers make. The proposed solution was then grounded in established ML theory, reframing it as a novel, inference-time application of cost-sensitive learning.

A separate body of literature was shown to provide a path forward, demonstrating the value of using retrieval confidence scores to calibrate downstream tasks (Tao et al., 2024; Chen et al., 2025). This review identifies a clear and novel research gap, confirmed by a systematic literature search (Table 2.1): no existing work bridges these two fields. The literature shows SOTA RAG retrievers are dynamic (e.g., SGIC), but SOTA verifiers are static. The proposed "Confidence-Weighted CONLI" algorithm aims to fill this gap, providing the foundational component for a new, more efficient class of adaptive verifiers.

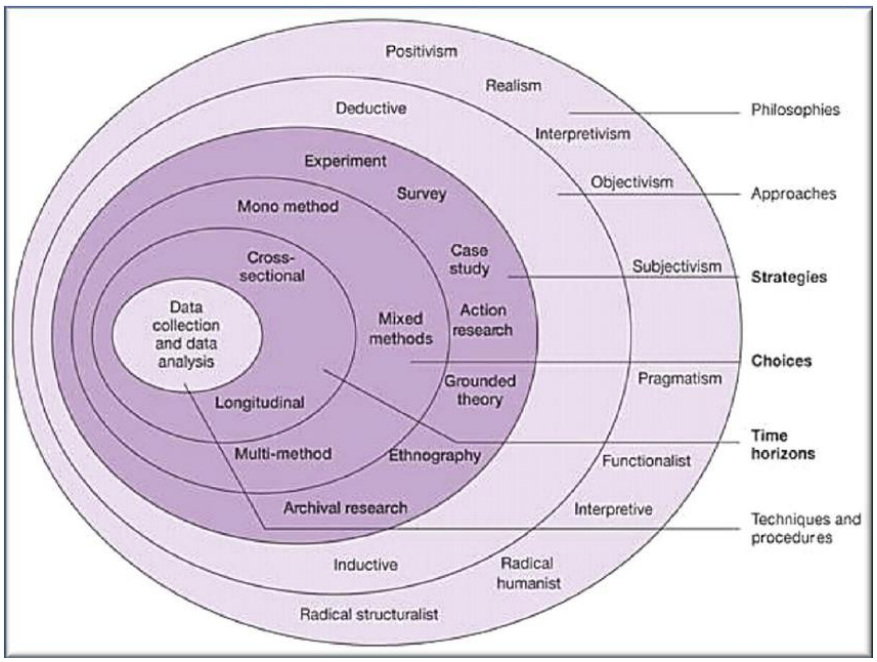
CHAPTER 03: METHODOLOGY AND PROJECT MANAGEMENT

3.1 Chapter overview

This chapter provides a detailed description of the research and development methodologies that will be employed to achieve the project's objectives. It begins by outlining the overarching research philosophy (Pragmatism) and approach (Deductive, Experimental). It then details the specific solution methodology, providing a technical blueprint for the new, streamlined two-layer "AFLHR Lite" pipeline. This includes the design of the novel Confidence-Weighted CONLI algorithm, illustrated with new architectural and logical diagrams, and the rigorous 3-condition evaluation methodology. Finally, the chapter concludes with a project management plan, including risk analysis and a clear statement of limitations.

3.2 Research methodology

The research methodology for this project is structured according to Saunders' Research Onion, providing a clear and justified framework for the research process.



Layer	Selection	Justification
Philosophy	Pragmatism	This project combines quantitative performance evaluation (F1-score, latency) of a machine learning pipeline with the practical goal of solving a real-world problem (hallucination). Pragmatism is suitable as it focuses on the research problem and uses the best-suited approaches to find a practical solution.
Approach	Deductive	The research begins with the

		established theory that uniform verification thresholds are suboptimal. It then develops a specific hypothesis: that a dynamic, confidence-weighted threshold will be more effective. This hypothesis will be tested through controlled experiments.
Methodological Choice	Mono Method (Quantitative)	The core of the thesis is the empirical evaluation of the AFLHR Lite pipeline. This involves collecting quantitative data (F1-score, Latency, Over-flagging Rate) from controlled experiments on a benchmark dataset.
Strategy	Experiment	The primary strategy is a controlled experiment. A baseline RAG-only condition (C1) and a control RAG + Standard CONLI (C2) will be compared against the experimental condition (C3: RAG + Confidence-Weighted CONLI) to isolate the effect of

		the intervention and test the research hypothesis.
Time Horizon	Cross-sectional	The data for model evaluation will be collected at a single point in time by running the models on the test benchmarks.
Data Collection	Secondary Data Analysis	The project will utilize the publicly available, pre-existing, human-annotated HaluEval dataset (Li et al., 2023) for evaluation.

Hypothesis: Dynamic retrieval confidence weighting of Natural Language Inference verification thresholds will produce statistically significant improvements in hallucination detection performance (≥5 percentage points in F1-score) compared to uniform, static thresholding approaches, while simultaneously reducing false positive rates by at least 10 percentage points and maintaining acceptable inference latency (≤20% overhead). This hypothesis is grounded in the core insight that the verification layer's blindness to evidence quality represents an exploitable opportunity; by instrumentalizing retrieval confidence scores already generated by the RAG component, the system can adaptively modulate verification strictness based on evidence trustworthiness, fundamentally addressing the one-size-fits-all limitation that plagues current implementations.

3.3 Development methodology

Prototyping is the selected development methodology for this research project. It is an iterative software development approach that emphasizes building early versions of a system to gather user

feedback and refine requirements, and it helps identify gaps and improve usability and clarify expectations before the final implementations. Considering its advantages, this methodology would be the most suitable for this project.

System Architecture & Implementation Setup: The core engine is implemented as the AFLHREngine class in engine.py, with configuration externalized to config.py and the user interface managed in app.py. Additionally, a diagnostic script (test_components.py) is included to test PyTorch, transformers, FAISS, and RoBERTa loading in isolation.

All models execute on the CPU to ensure cross-platform stability. GPU/MPS acceleration was deferred due to Apple Silicon compatibility issues with the PyTorch MPS backend. To further guarantee stability, environment variables (PYTORCH_ENABLE_MPS_FALLBACK, TOKENIZERS_PARALLELISM, OMP_NUM_THREADS, MKL_NUM_THREADS) are explicitly hardened prior to execution.

3.3.1 Requirement elicitation methodology

Out of the various methodologies available for requirements specification, the below were selected as they were deemed best suited for this project.

3.3.2 Design methodology

OOADM (Object-Oriented Analysis and Design Methodology) has been selected as the design methodology for this project, as it aligns well with the modular and iterative nature of

the proposed system, since the project involves multiple interacting components, such as agents, data retrieval modules, and analytical services.

3.3.3 Programming paradigm

This implementation follows a hybrid programming paradigm with a combination of Object-Oriented Programming (OOP), Structured programming, and functional programming principles.

• **OOP**: Applied primarily in the system orchestration layer (LangChain-based pipeline management) and the modular verification layer architecture, enabling clean separation between the RAG retrieval module and the CONLI verification module.

• **Structured Programming**: Adopted for procedural operations involving data ingestion, vector database querying (FAISS), and the sequential processing of premise-hypothesis pairs through the NLI classifier, contributing mainly to pipeline reproducibility and deterministic verification logic.

• **Functional Programming**: Applied within the confidence-weighting algorithm, the dynamic threshold computation functions, and the cost-sensitive learning components that map retrieval scores to NLI thresholds, enabling composable, testable verification operations.

• **RAG Retrieval Layer Specifications:** Embeddings are generated using transformers.AutoModel (model ID: sentence-transformers/all-MiniLM-L6-v2) utilizing manual mean pooling over token embeddings, excluding padding tokens, followed by L2 normalization. This approach functionally replicates SentenceTransformer behavior while avoiding library-specific compatibility issues. The index building process is managed via the ingest_knowledge_base() method. The default retrieval operation (retrieve()) returns the top-2 documents (k=2). The retrieval confidence score is computed using the formula (raw_cosine_similarity + 1) / 2, which normalizes the output from [-1, 1] to [0, 1]. The retrieve() method returns a comprehensive dictionary containing: context (concatenated document text), retrieval_score (normalized 0-1), raw_score (cosine similarity), documents (list of individual documents), and indices (FAISS index positions).

• **Generative Layer Specifications**: The primary LLM generation step uses a strict system prompt: "You are a precise assistant. Answer the question using ONLY the context provided. Be concise." To ensure near-deterministic outputs, a temperature setting of 0.1 is employed. Furthermore, the generative phase includes robust error handling that returns an error string on exception rather than crashing the pipeline.

• **Verification Layer (CONLI) Specifications:** he Natural Language Inference (NLI) step uses truncation='only_first' to handle long inputs: the premise (retrieved context) is truncated as needed, while the hypothesis (generated response) is preserved intact, ensuring the claim being verified is always fully evaluated. The pretrained RoBERTa-MNLI model outputs logits in the order [contradiction, neutral, entailment]. The entailment probability (index 2) is specifically extracted after softmax normalization.

Algorithmic Pseudocode for Confidence-Weighted CONLI

```
Algorithm: Confidence-Weighted CONLI (Calculate Verdict)
Input: retrieval_score (float), nli_entailment_score (float), pivot (float), strict_threshold
(float), lenient_threshold (float)
Output: verdict_dict (dict)

1. IF retrieval_score < pivot THEN
     mode = "STRICT"
     active_threshold = strict_threshold
   ELSE
     mode = "LENIENT"
     active_threshold = lenient_threshold

2. IF nli_entailment_score >= active_threshold THEN
     status = "VERIFIED"
     passed = True
   ELSE
     status = "HALLUCINATION"
     passed = False

3. Return dictionary: {
     status: status,
     mode: mode,
     threshold: active_threshold,
     nli_score: nli_entailment_score,
     retrieval_score: retrieval_score,
     reasoning: [generated explanation string],
     passed: passed
   }
```

3.3.4 Evaluation methodology

A rigorous empirical experiment is designed to test the project's hypothesis (RO3).

- **Dataset**: The HaluEval benchmark (Li et al., 2023). The human-annotated QA dataset will be split into a dev set (for tuning) and a test set (for final evaluation). Using a dev set for tuning NLI thresholds is a standard practice.
- **Threshold Tuning**: A grid search will be performed on the dev set to find the optimal parameters for C2 and C3.
  - For C2 (Standard CONLI), the single T_static threshold will be tuned to maximize F1-score.
  - For C3 (Weighted CONLI), the parameters (RETRIEVAL_HIGH_CONF, T_LENIENT, T_STRICT, etc.) will be tuned to maximize F1-score.
- **Experimental Conditions**: The "locked-in" models will be run once on the blind test set to generate final, publishable results.

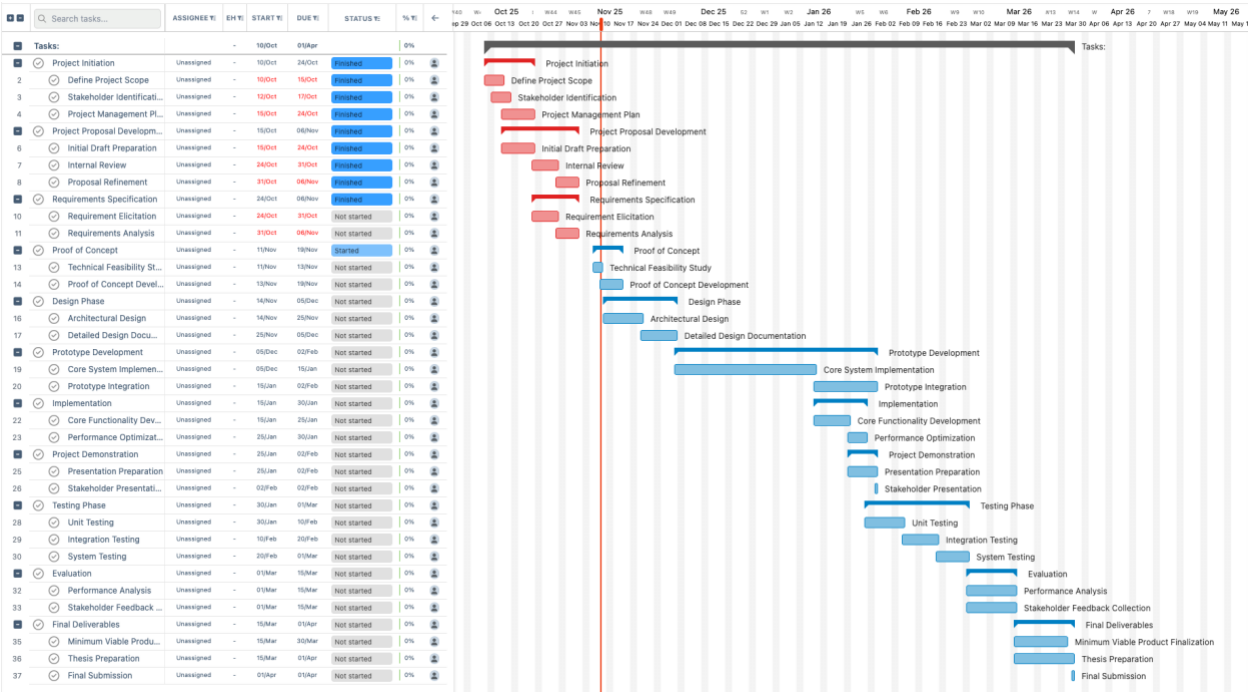
Key Metrics: (RO4)

1. Hallucination Detection F1-Score: The primary metric. F1 = 2 \times (Precision \times Recall) / (Precision + Recall), where:
   - **True Positive (TP)**: A hallucination is correctly flagged as REJECTED.
   - **False Positive (FP):** A valid sentence is incorrectly flagged as REJECTED.
   - False Negative (FN): A hallucination is incorrectly VERIFIED.
2. Latency: Average time (in milliseconds) per query across the test set.
3. Over-Flagging Rate (False Positive Rate): Measures the rate of false positives against all valid sentences. Over-Flagging Rate = FP / (FP + TN) (where TN is a correctly verified valid sentence). This quantifies how often the verifier rejects good content, a key metric in RAG evaluation (Khasanova Zafar kizi & Suh, 2025).

3.4 Project management methodology

An Agile approach was used to manage this project, enabling iterative development and flexibility during implementation. This methodology allows continuous refinement of the verification framework, RAG pipeline integration, and evaluation components.

3.4.1 Gantt chart



3.4.2 Deliverables

Deliverables	Dates

Project Proposal and Requirement Specification – Initial draft	24th Oct 2025
Project Proposal and Requirement Specification – Final	13th Nov 2025
Proof of Concept	19th Nov 2025
Design Phase	5th Dec 2025
Prototype	2nd Feb 2026
Interim Project Demo	2nd Feb 2026
Implementation	30th Jan 2026
Testing	1st March 2026
Evaluation	1st March 2026
Thesis Submission	1st Apr 2026
Minimum Viable Product	1st Apr 2026

3.4.3 Resource requirements

3.4.3.1 Skills

- Machine Learning and Data Science Expertise – Since the project involves the development and evaluation of retrieval-augmented generation (RAG) and cost optimization models, a good

understanding of machine learning, data processing, and evaluation metrics is essential

- Python Programming Skills – The system implementation, including the agent orchestration layer, data pipelines, and integration with APIs and vector databases, relies on Python.
- System Design and Integration Skills – Knowledge of system design, API integration, and architectural diagramming is needed.
- Academic Writing and Communication Skills – Strong writing and presentation skills are necessary for documenting research findings, preparing the final thesis, and presenting the research.

3.4.4 Risk and mitigation (Expanded with Severity and Frequency)

Risk	Severity	Frequency	Mitigation Strategy
Optimal Threshold Tuning and Model Convergence	5	3	Test both tiered-threshold model and continuous weighting function (Step 2B); perform extensive grid search on dev set; investigate alternative weighting schemes if convergence fails.
Noisy and Poorly-Calibrated Retrieval Scores	3	3	Analyze cosine similarity score distributions; implement score normalization techniques; validate retrieval quality against ground truth; consider alternative embedding models if necessary.
Computational Resource Constraints and GPU Memory Management	3	1	Leverage scalable cloud infrastructure (Google Colab Pro, AWS EC2); implement model quantization (bf16 precision); use gradient checkpointing to reduce memory footprint.

Data Quality and HaluEval Dataset Limitations	3	2	Validate dev set quality; cross-check annotations; consider supplementary datasets if HaluEval proves insufficient; document any limitations in evaluation results.
NLI Model Overfitting to Development Set	5	3	Use separate blind test set for final evaluation; implement early stopping during tuning; validate hyperparameters across multiple random seeds; monitor generalization gaps.
Latency Overhead Exceeding 20% Threshold	4	2	Profile each component (retrieval, generation, verification); optimize NLI inference with quantization or batch processing; consider lighter NLI models if necessary.
Integration Issues Between RAG and CONLI Components	3	2	Conduct unit testing for each layer; establish clear data contracts between Layer 1 and Layer 2; implement comprehensive logging and error handling.
Insufficient Improvement Over Static Baseline	5	2	Document all negative results as valid research outcomes; analyze failure modes; publish findings regardless of outcome; consider alternative adaptive strategies.

3.4.4 Chapter summary

In conclusion, this chapter outlines the comprehensive research and development approach for implementing the AFLHR Lite two-layer hallucination mitigation pipeline. approach. The hybrid programming paradigm enables seamless integration across the analytical and orchestration levels,

the prototyping and OOADM approaches guarantee modularity and adaptability.

CHAPTER 04: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

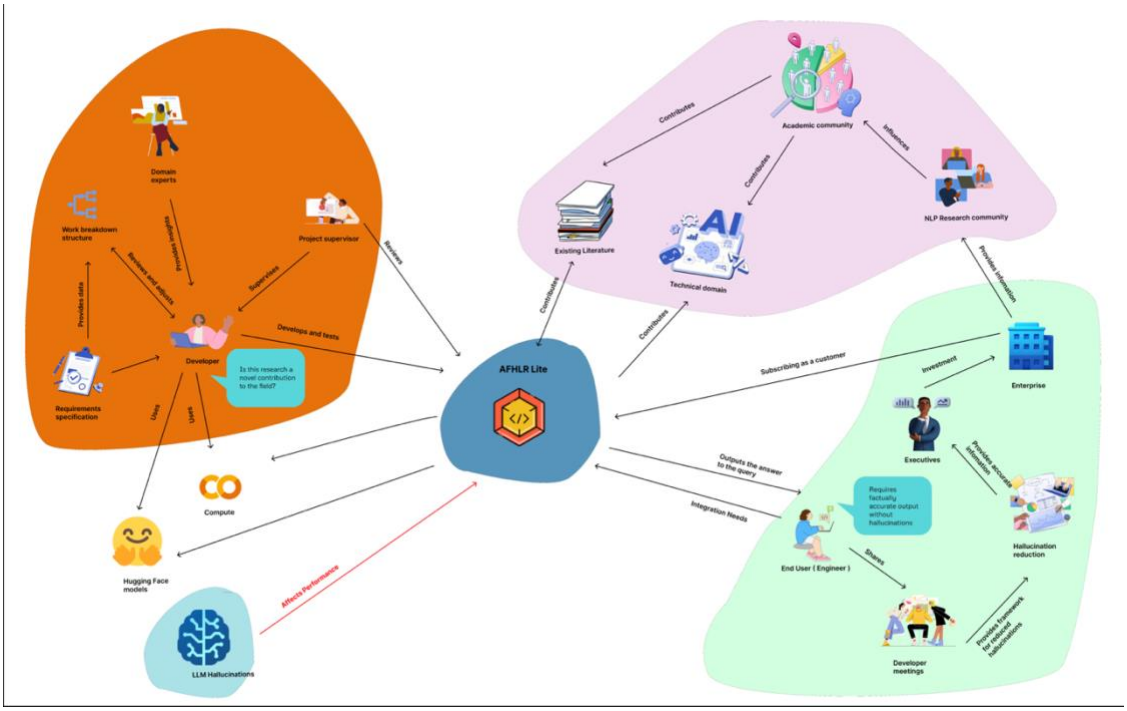
4.1 Chapter overview

The Software Requirement Specification (SRS) chapter provides a detailed analysis of the system's requirements, revised to align with the new "AFLHR Lite" scope. It identifies stakeholders, outlines elicitation methods, and uses visual modeling concepts to illustrate the system's interactions. The core of the chapter is the detailed list of functional and non-functional requirements, which are prioritized using the MoSCoW principle.

4.2 Rich picture diagram

The Rich Picture Diagram (Figure 4.1) illustrates the key stakeholders, processes, and concerns involved in the AFLHR Lite project. This non-technical diagram provides a high-level, contextual overview of the problem space.

Figure 4.1: Rich Picture Diagram



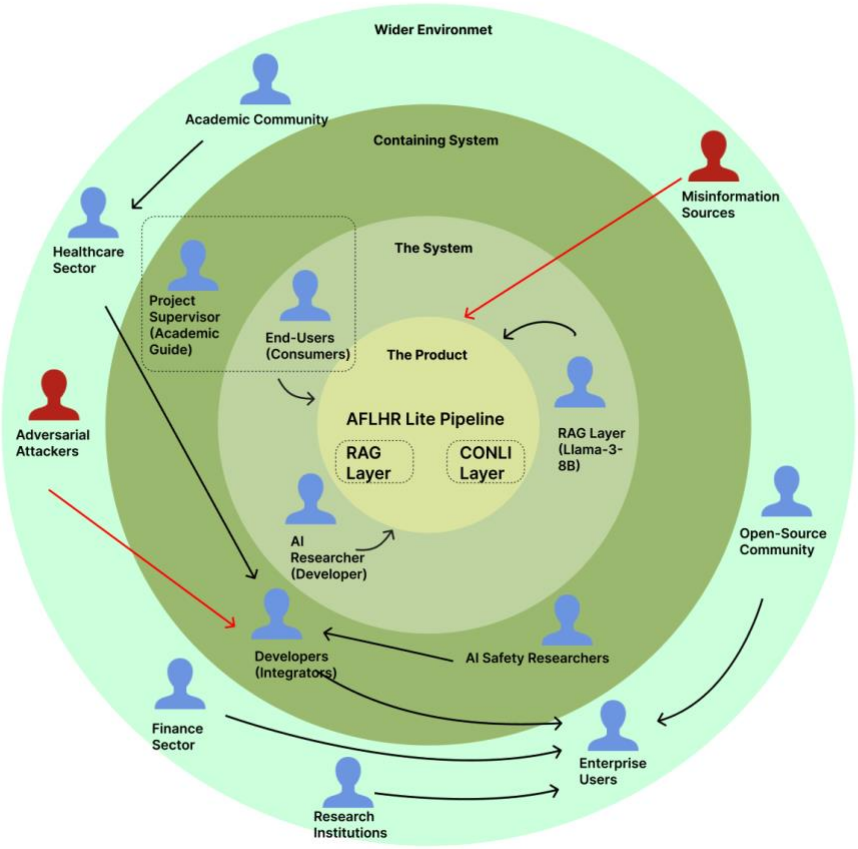
This diagram captures the flow from external knowledge sources (Wikipedia) through the two-layer system, managed by the AI Researcher, to produce a verified response. It highlights the primary concerns of latency and complexity against the key goals of measurable F1-score improvement and reduced over-flagging.

4.3 Stakeholder analysis

4.3.1 Stakeholder Onion Model

The stakeholders of this research project remain the same as in the original proposal, as their core requirements are unchanged by the re-scoping.

Table 4.1: Stakeholder Onion Model



4.3.2 Stakeholder Viewpoints

Stakeholder	Stakeholder	Stakeholder
Layer 1: The Product		
RAG Layer (Llama-3-8B)	Core Component (Generator)	The generative LLM (meta-llama/Meta-Llama-3-8B-Instruct) specified as the core component for producing the initial "hypothesis" (response). It is also the source of the "extrinsic hallucinations" (unfaithfulness to provided source material) that the system is designed to mitigate.
CONLI Layer	Core	The programmatic "Confidence-Weighted CONLI"

	Component (Verifier)	layer. This component is the core innovation, using a specialized, objective NLI classifier (FacebookAI/roberta-large-mnli) to act as the "objective arbiter" that checks the hypothesis against the retrieved premise. It is the "enforcer" of the adaptive verification rules.
Layer 2: The System		
End-Users (Consumers)	Primary Beneficiary	The direct consumers of the LLM's output, as identified in the SRS. Their primary concern is *trust* and *reliability*. They require factually accurate and logically coherent information, as a "single hallucination... could have severe consequences". They are also the stakeholder most damaged by the "over-flagging" (false positive) problem, which "degraded user experience".
AI Researcher (Developer)	System Architect & Evaluator	The *builder* of the AFLHR Lite pipeline and the *author* of the research. Their primary concern is *technical and academic novelty*. They must design a modular, testable system (NFR02) and produce *quantifiable results* that demonstrate a *statistically significant improvement* over baselines (RO3: "Achieve F improvement").
Developers (Integrators)	System Implementer	The technical team responsible for deploying the AFLHR Lite pipeline into broader, real-world applications. Their primary concern is *practicality, performance, and cost-effectiveness*. They require a "plug-and-play" module that manages the "accuracy-latency trade-off." The *latency budget* (RO2: "Latency increase") is their key non-functional requirement.
AI Safety	External	Members of the broader research community

Researchers	Auditor / Domain Expert	(distinct from the internal *AI Researcher*) whose *role* is to study, quantify, and mitigate AI risks. Their *concern* is the *systemic risk* of LLMs propagating "misinformation, safety, and reliability" issues. This project *contributes directly to* their research domain by offering a novel, robust verification method that addresses a known failure mode of existing systems.
End-Users (Consumers)	Primary Beneficiary	The direct consumers of the LLM's output, as identified in theSRS. Their primary concern is *trust* and *reliability*. They require factually accurate and logically coherent information, as a "single hallucination... could have severe consequences". They are also the stakeholder most damaged by the "over-flagging" (false positive) problem, which "degraded user experience".
AI Researcher (Developer)	System Architect & Evaluator	The *builder* of the AFLHR Lite pipeline and the *author* of the research. Their primary concern is *technical and academic novelty*. They must design a modular, testable system (NFR02) and produce *quantifiable results* that demonstrate a *statistically significant improvement* over baselines (RO3: "Achieve F1 improvement").
Developers (Integrators)	System Implementer	The technical team responsible for deploying the AFLHR Lite pipeline into broader, real-world applications. Their primary concern is *practicality, performance, and cost-effectiveness*. They require a "plug-and-play" module that manages the "accuracy-latency trade-off." The *latency budget* (RO2: "Latency increase is their key non-functional requirement.

AI Safety Researchers	External Auditor / Domain Expert	Members of the broader research community (distinct from the internal *AI Researcher*) whose *role* is to study, quantify, and mitigate AI risks. Their *concern* is the *systemic risk* of LLMs propagating "misinformation, safety, and reliability" issues. This project *contributes directly to* their research domain by offering a novel, robust verification method that addresses a known failure mode of existing systems.
End-Users (Consumers)	Primary Beneficiary	The direct consumers of the LLM's output, as identified in the SRS. Their primary concern is *trust* and *reliability*. They require factually accurate and logically coherent information, as a "single hallucination... could have severe consequences". They are also the stakeholder most damaged by the "over-flagging" (false positive) problem, which "degraded user experience".
AI Researcher (Developer)	System Architect & Evaluator	The *builder* of the AFLHR Lite pipeline and the *author* of the research. Their primary concern is *technical and academic novelty*. They must design a modular, testable system (NFR02) and produce *quantifiable results* that demonstrate a *statistically significant improvement* over baselines (RO3: "Achieve F1 improvement").
Developers (Integrators)	System Implementer	The technical team responsible for deploying the AFLHR Lite pipeline into broader, real-world applications. Their primary concern is *practicality, performance, and cost-effectiveness*. They require a "plug-and-play" module that manages the "accuracy-latency trade-off." The *latency budget* (RO2: "Latency increase is their key non-functional

		requirement.[1]
Layer 3: The Containing System		
Project Supervisor (Academic Guide)	Academic & Project Authority	The stakeholder responsible for guiding the research and ensuring its quality. Their primary concern is *academic rigor and novelty*. They require the system to address a "critical, un-bridged research gap" and be "rigorously evaluated" against SOTA baselines in a 3-condition experiment.
Healthcare Sector	High-Stakes Adopter	A critical domain where "factual accuracy is paramount". Their concern is *patient safety and trust*. They have a *zero-tolerance* policy for hallucinations, as "a single hallucination in a medical diagnosis system could have severe consequences". They represent the stakeholder demand for the *strictest possible* verification.
Finance Sector	High-Stakes Adopter	A critical, "regulated" domain with concerns parallel to Healthcare. They require *auditable, reliable, and factually-sound* outputs to prevent financial errors, maintain regulatory compliance, and uphold user trust. They share the "zero-tolerance" policy for factual inaccuracies.
Enterprise Users	Operational Adopter	Regulated industries and enterprise systems seeking to deploy LLMs for *efficiency gains*. Their primary concerns are *accountability, auditability,* and *cost-effectiveness*. They are the stakeholder *most* impacted by "over-flagging" (false positives), which creates a "verification burden" that "negates potential efficiency gains".
Research Institutions	Funder / Host Organization	The university or lab (as shown in the diagram) that hosts and supports the project. Their concern is the *production of novel research* and *contributions to*

		the field. They are a key beneficiary of the project's "public release" of the core algorithm and its contribution to the *research domain* , which enhances the institution's reputation.
Open-Source Community	Community Collaborator & Adopter	The external ecosystem of developers who will potentially use, fork, and build upon the project. Their concern is *accessibility, reproducibility, and modularity*. The project *directly* serves them by using an open-source stack (LangChain, FAISS, Llama-3, RoBERTa) and planning the "public release of the Confidence-Weighted CONLI algorithm".
Project Supervisor (Academic Guide)	Academic & Project Authority	The stakeholder responsible for guiding the research and ensuring its quality. Their primary concern is *academic rigor and novelty*. They require the system to address a "critical, un-bridged research gap" and be "rigorously evaluated" against SOTA baselines in a 3-condition experiment.
Healthcare Sector	High-Stakes Adopter	A critical domain where "factual accuracy is paramount" Their concern is *patient safety and trust*. They have a *zero-tolerance* policy for hallucinations, as "a single hallucination in a medical diagnosis system could have severe consequences". They represent the stakeholder demand for the *strictest possible* verification.
Finance Sector	High-Stakes Adopter	A critical, "regulated" domain with concerns parallel to Healthcare. They require *auditable, reliable, and factually-sound* outputs to prevent financial errors, maintain regulatory compliance, and uphold user trust. They share the "zero-tolerance" policy for factual inaccuracies.

Enterprise Users	Operational Adopter	Regulated industries and enterprise systems seeking to deploy LLMs for *efficiency gains*. Their primary concerns are *accountability, auditability,* and *cost-effectiveness*. They are the stakeholder *most* impacted by "over-flagging" (false positives), which creates a "verification burden" that "negates potential efficiency gains".
Research Institutions	Funder / Host Organization	The university or lab (as shown in the diagram) that hosts and supports the project. Their concern is the *production of novel research* and *contributions to the field*. They are a key beneficiary of the project's "public release" of the core algorithm and its contribution to the *research domain* , which enhances the institution's reputation.
Open-Source Community	Community Collaborator & Adopter	The external ecosystem of developers who will potentially use, fork, and build upon the project. Their concern is *accessibility, reproducibility, and modularity*. The project *directly* serves them by using an open-source stack (LangChain, FAISS, Llama-3, RoBERTa) and planning the "public release of the Confidence-Weighted CONLI algorithm".

4.4 Selection of Requirement Elicitation Methodologies

For this project, various methodologies were employed to gather requirements. Each methodology was selected based on its ability to capture relevant, accurate, and actionable insights from stakeholders.

Method 1: Literature Review
A systematic and comprehensive literature review served as the foundational elicitation method that involved the analysis of peer-reviewed publications, conference proceedings, and technical documentation across three domains: (1) hallucination detection mechanisms in LLMs, (2) Retrieval-Augmented Generation architectures and their confidence calibration methods, and (3) Natural Language Inference techniques for factuality verification. Furthermore, the literature review provides an overview of the limitations within the solutions already existing.
Method 2: Prototyping
An iterative approach used to design and develop the proposed system prototype. This includes implementing the RAG workflow and other tests. With the methodology being prototyping, the goal here is to ensure that the best system development decisions, are based on real data gathered by prototyping.

4.5 Discussion of findings

4.5.1 Literature review findings

Below are detailed overviews of the findings that were realized after conducting a thorough literature review. These contributed to identifying and outlining the system requirements for the proposed system.

Finding 1: Verification Systems Are Rigid and Operate in Isolation
Theme: Architectural Disconnect Between Retrieval and Verification
Citation: (Lei et al., 2023)

Current LLM verifiers rely on inflexible, uniform thresholds without integrating evidence quality signals from the retrieval layer. RAG systems produce numerical confidence scores (e.g., cosine similarity), but these are typically discarded before verification, meaning the system treats claims supported by high-confidence evidence (0.99) identically to those with low-confidence evidence (0.65). This architectural disconnect represents a missed opportunity for adaptive verification.

Finding 2: Two Distinct Research Areas Remain Un-Bridged
Theme: Research Gap Between Hallucination Detection and Retrieval Calibration
Citation: (Lei et al., 2023; Tao et al., 2024; Chen et al., 2025)

Hallucination Verification research (CONLI, CoVe, SelfCheckGPT) and Retrieval Calibration research (confidence-based modulation) exist as separate fields. Recent work shows retrieval confidence scores reliably predict output trustworthiness, yet no published framework integrates these by using retrieval confidence to dynamically modulate NLI verifier thresholds. This gap is the core motivation for this research.

Finding 3: Retrieval Confidence Scores Are Typically Discarded
Theme: Under-Utilized Proxy for Evidence Trustworthiness
Citation: (Lewis et al., 2020; Douze et al., 2025)

RAG systems produce numerical confidence scores as a standard byproduct (e.g., cosine similarity 0.0-1.0), representing a quantitative proxy for evidence quality. However, current pipelines extract only the retrieved text and discard the score—losing a computationally free, actionable signal that directly reflects evidence reliability.

Finding 4: NLI Verification Suffers from the Over-Flagging Problem
Theme: The Unresolved Precision-Recall Trade-Off
Citation: (Lei et al., 2023; Wang et al., 2025)

NLI-based verifiers (e.g., CONLI) apply a single, rigid entailment threshold. A strict threshold (0.99) catches hallucinations but over-flags valid, nuanced responses; a lenient threshold (0.85) improves coverage but misses hallucinations. The literature treats this as inevitable, yet it can be solved through adaptive, evidence-aware thresholding.

Finding 5: Specialized NLI Classifiers Are Superior to Self-Verification
Theme: Objective Classifiers vs. Self-Biased LLMs
Citation: (Liu et al., 2019; Williams et al., 2018; Kryscinski et al., 2020)

Specialized NLI models (e.g., roberta-large-mnli trained on MultiNLI) provide objective, non-hallucinating verification superior to self-verification methods, which suffer from circular reasoning. These classifiers are reliable arbiters for the verification layer without self-bias.

Finding 6: Static Thresholds Persist Due to Simplicity-Accuracy Trade-Offs
Theme: Engineering Rationale for Current Approaches
Citation: (Lei et al., 2023; Wang et al., 2025)

Static thresholds are computationally cheap and simple to implement, making them rational for non-critical applications. However, high-stakes domains (healthcare, finance) require higher accuracy. The project addresses this by introducing adaptive logic without prohibitive latency costs.

Finding 7: Cost-Sensitive Learning Provides Theoretical Foundation
Theme: ML Theory for Addressing Over-Flagging
Citation: (Domingos, 1999; Lei et al., 2023)

Cost-sensitive learning treats misclassification errors as having unequal costs. Confidence-Weighted CONLI uses retrieval_score as a dynamic cost proxy: low scores trigger strict thresholds (0.99) to avoid accepting hallucinations; high scores trigger lenient thresholds (0.85) to avoid over-flagging. This is a principled ML application, not an ad hoc heuristic.

Finding 8: HaluEval Benchmark Provides Rigorous Evaluation Foundation

Theme: Human-Annotated Ground Truth for Hallucination Detection

Citation: (Li et al., 2023)

HaluEval is a large-scale, human-annotated benchmark specifically designed for hallucination evaluation, providing diverse hallucination types and ground truth for rigorous assessment of verification approaches.

4.5.2 Prototype findings

Retrieval Score Distribution Is Non-Uniform
During prototyping, I discovered that retrieval scores from the embedding model (all-MiniLM-L6-v2) cluster heavily in narrow ranges rather than distributing uniformly across 0.0-1.0. For example, most scores might fall between 0.75-0.95, with very few below 0.60. This skewed distribution means the "low confidence" tier (e.g., scores < 0.70) could receive no real-world data, making your tiered thresholds ineffective or poorly calibrated.
The NLI Model Sometimes Abstains on Ambiguous Claims
The RoBERTa-large-MNLI classifier produced entailment scores that are genuinely ambiguous (e.g., 0.50 ± 0.05 across Entailment/Neutral/Contradiction). This occurs with nuanced claims that don't clearly follow from or contradict the premise.

4.6 Summary of Findings

ID	Findings	Literature Review	Prototyping
F1	Verification systems rely on inflexible, uniform thresholds without integrating evidence quality signals.	✓	
F2	No published framework integrates hallucination verification with retrieval calibration using confidence to modulate NLI thresholds.	✓	
F3	RAG systems produce numerical confidence scores that are typically discarded in the verification layer.	✓	
F4	NLI-based verifiers suffer from over-flagging: strict	✓	

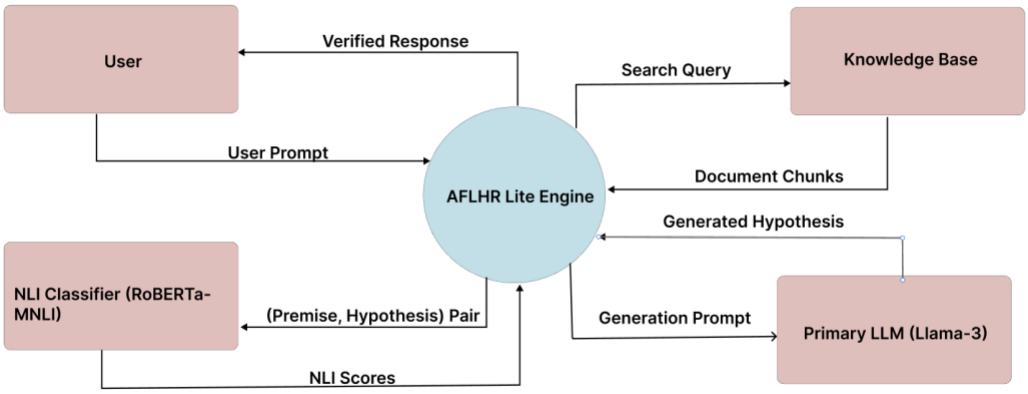
	thresholds reject valid responses; lenient thresholds miss hallucinations.		
F5	Retrieval score distributions may be non-uniform, affecting tiered threshold calibration.		✓

4.6 Context diagram

The Level 0 Data Flow Diagram (Context Diagram) in Figure 4.2 defines the system's boundary and its interactions with external entities.

Figure 4.2: Context Diagram

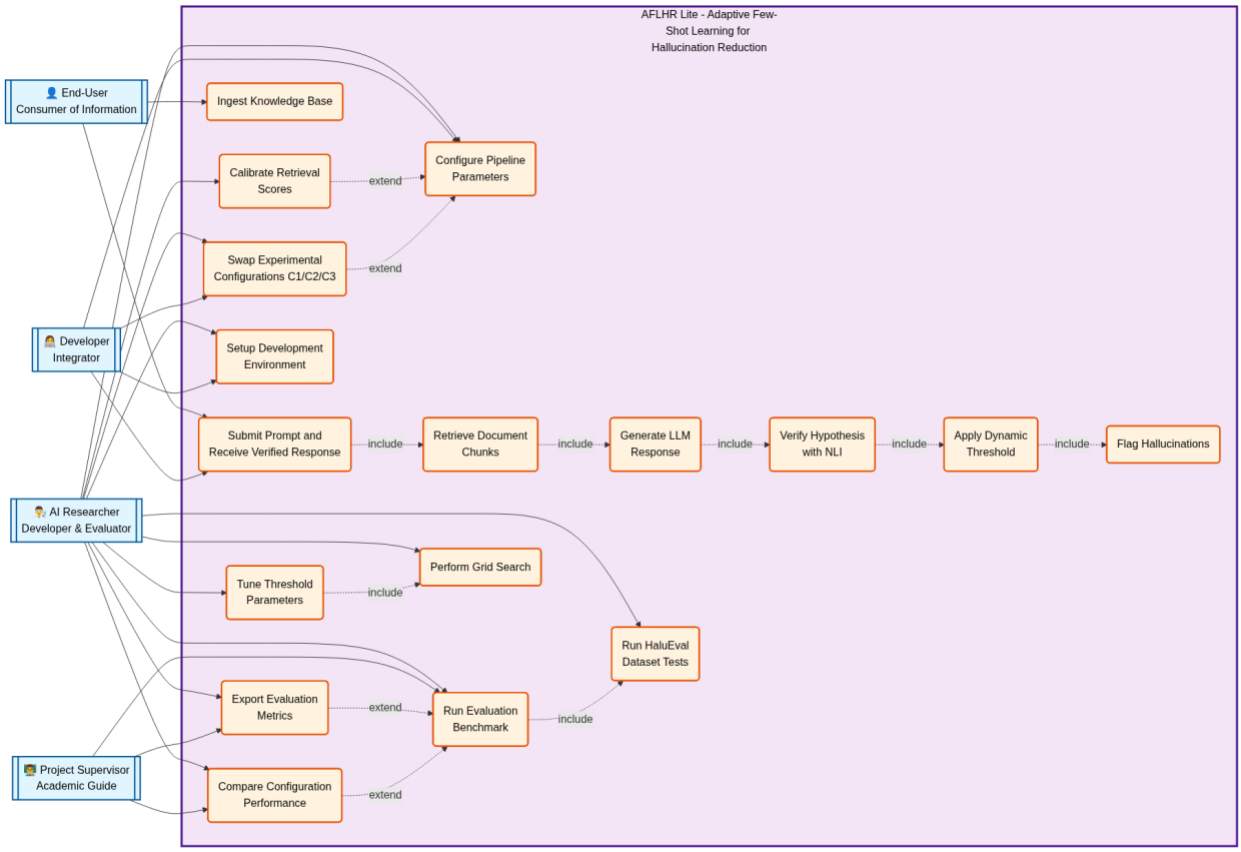
This diagram shows the AFLHR Lite Engine as the central process. It interacts with four external entities: the User (providing prompts, receiving responses), the Knowledge Base (providing documents), the Primary LLM (generating the hypothesis), and the NLI Classifier (providing objective scores).



4.7 Use case diagram

The Use Case Diagram in Figure 4.3 specifies the high-level functionalities of the system and the users who interact with them.

Figure 4.3: Use Case Diagram



4.9 Use case descriptions

Use case	Submit Prompt and Receive Verified Response
Description	This use case describes the primary function of the AFLHR Lite system. A user inputs a query, which prompts the system to execute its full, multi-layer pipeline.
Actors	End-User, Developer / Integrator, AI Researcher
Extended use-cases	N/A

Included use-cases	1. Retrieve Document Chunks 2. Generate LLM Response 3. Apply Hypothesis w/ NLI 4. Apply Dynamic Threshold
Main flow	Actor: The user (End-User, Developer, or Researcher) submits a prompt to the system.

4.8 Requirements

The MoSCoW principle is used to prioritize requirements into:

• M: Must Have (mandatory).

• S: Should Have (important).

• C: Could Have (desirable).

• W: Will Not Have (out of scope).

4.8.1 Functional Requirements (Fr)

Table 4.2: Functional Requirements

ID	Priority Level	Requirement Description

FR01	MUST	The system must ingest a knowledge base and user query.
FR02	MUST	The RAG system must retrieve semantically relevant document chunks.
FR02.1	MUST	The RAG retriever must output both the document chunk (premise) and its numerical retrieval_score (Douze et al., 2025).
FR03	MUST	The system must use the Primary LLM (Llama-3-8B) to generate an initial response (hypothesis) based on the retrieved premise.
FR04	MUST	The system must verify the hypothesis against the premise using a pretrained NLI classifier (roberta-large-mnli) (Liu et al., 2019).
FR05	MUST	The system must implement the Confidence-Weighted CONLI algorithm (as defined in Table 3.2).
FR05.1	MUST	The system must apply a dynamic NLI entailment threshold based on the retrieval_score from FR02.1.
FR06	MUST	The system must flag any sentence that fails the dynamic verification check as a REJECTED_AS_HALLUCINATION.
FR07	SHOULD	The system should provide a programmatic interface to run the evaluation harness (C1, C2, C3) on the HaluEval dataset (Li et al., 2023).
FR08	COULD	The system could implement an alternative continuous weighting

		function (as defined in Section 3.3.3, Step 2B) to compare against the tiered-threshold model.
FR09	WILL NOT HAVE	The system will not implement a cross-model consistency check (Deferred to future work).
FR10	WILL NOT HAVE	The system will not implement a surgical correction layer to rewrite flawed sentences (Deferred to future work).
FR11	WILL NOT HAVE	The system will not implement a DPO pipeline (Deferred to future work).

4.8.2 Non-Functional Requirements

Table 4.3: Non-Functional Requirements

ID	Priority Level	Requirement and Description
NFR01	MUST	(Reliability) The system must produce deterministic verification outputs for the same (premise, hypothesis, score) triplet.
NFR02	MUST	(Modularity) The verification layer (Layer 2) must be modular, allowing the Standard-CONLI (C2) and Confidence-Weighted-CONLI (C3) components to be swapped for evaluation.
NFR03	MUST	(Performance) The system's performance (F1, Latency,

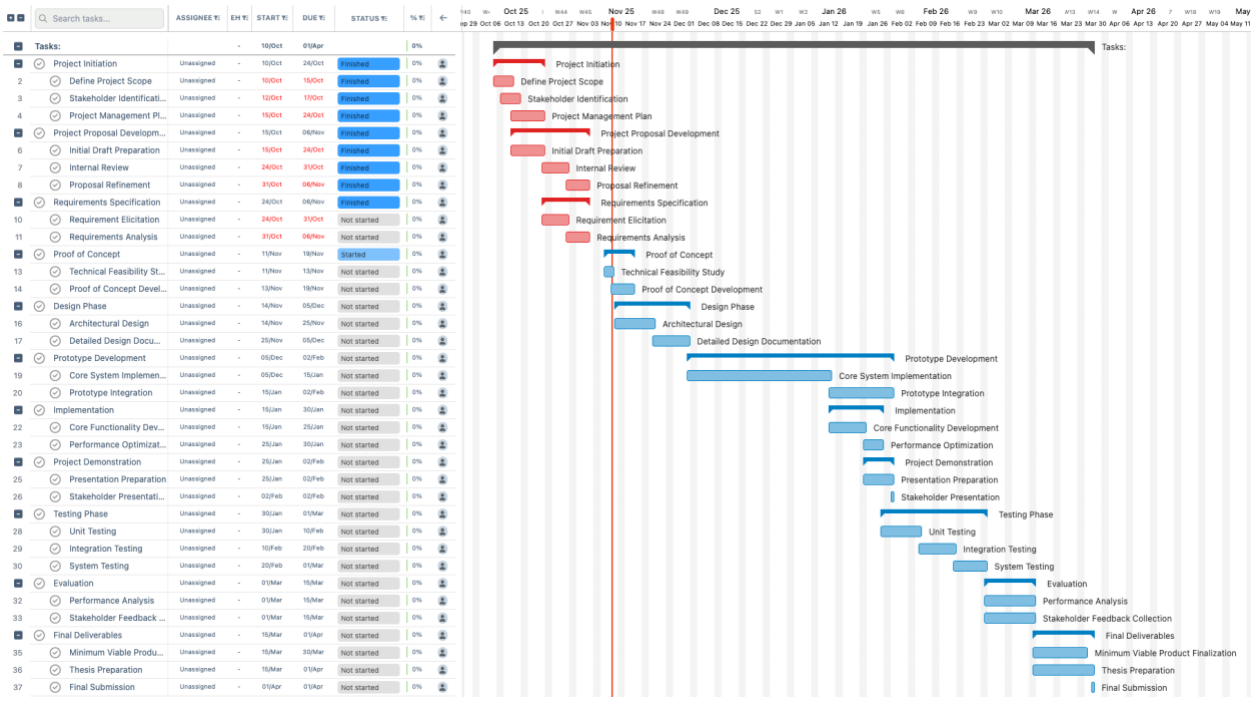
		Over-flagging) must be quantifiable on the HaluEval benchmark (Li et al., 2023).
NFR04	SHOULD	(Usability) The evaluation harness must be executable via a single script.
NFR05	MUST	(Resource Efficiency) (Revised) The full end-to-end inference pipeline (AFLHR Lite) must be able to run on a single GPU with < 20GB VRAM. (This requirement is reduced from the original proposal, as the 4-bit Mistral-7B model is no longer in scope).

4.8.3 Chapter summary

This chapter presents the methodology used for the identification, analysis, and documentation of system requirements. The process commenced with a thorough stakeholder analysis, utilizing the Rich Picture Diagram and Stakeholder Onion Model to map influence and perspectives. Subsequently, targeted elicitation techniques were employed to gather requirements systematically. System boundaries and interactions were then defined using Context and Use Case Diagrams. Finally, the chapter provides a comprehensive specification of functional and non-functional requirements, which have been prioritized according to the MoSCoW framework.

CHAPTER 05: TIME SCHEDULE

5.1 Project schedule (Gantt chart)



	Tasks:	ASSIGNEE	EH	START	DUE	STATUS	%
	Tasks:		-	10/Oct	01/Apr		0%
	Project Initiation	Unassigned	-	10/Oct	24/Oct	Finished	0%
2	Define Project Scope	Unassigned	-	10/Oct	15/Oct	Finished	0%
3	Stakeholder Identificati...	Unassigned	-	12/Oct	17/Oct	Finished	0%
4	Project Management Pl...	Unassigned	-	15/Oct	24/Oct	Finished	0%
	Project Proposal Developm...	Unassigned	-	15/Oct	06/Nov	Finished	0%
6	Initial Draft Preparation	Unassigned	-	15/Oct	24/Oct	Finished	0%
7	Internal Review	Unassigned	-	24/Oct	31/Oct	Finished	0%
8	Proposal Refinement	Unassigned	-	31/Oct	06/Nov	Finished	0%
	Requirements Specification	Unassigned	-	24/Oct	06/Nov	Finished	0%
10	Requirement Elicitation	Unassigned	-	24/Oct	31/Oct	Not started	0%
11	Requirements Analysis	Unassigned	-	31/Oct	06/Nov	Not started	0%
	Proof of Concept	Unassigned	-	11/Nov	19/Nov	Started	0%
13	Technical Feasibility St...	Unassigned	-	11/Nov	13/Nov	Not started	0%
14	Proof of Concept Devel...	Unassigned	-	13/Nov	19/Nov	Not started	0%
	Design Phase	Unassigned	-	14/Nov	05/Dec	Not started	0%
16	Architectural Design	Unassigned	-	14/Nov	25/Nov	Not started	0%
17	Detailed Design Docu...	Unassigned	-	25/Nov	05/Dec	Not started	0%
	Prototype Development	Unassigned	-	05/Dec	02/Feb	Not started	0%
19	Core System Implemen...	Unassigned	-	05/Dec	15/Jan	Not started	0%
20	Prototype Integration	Unassigned	-	15/Jan	02/Feb	Not started	0%
	Implementation	Unassigned	-	15/Jan	30/Jan	Not started	0%
22	Core Functionality Dev...	Unassigned	-	15/Jan	25/Jan	Not started	0%
23	Performance Optimizat...	Unassigned	-	25/Jan	30/Jan	Not started	0%
	Project Demonstration	Unassigned	-	25/Jan	02/Feb	Not started	0%
25	Presentation Preparation	Unassigned	-	25/Jan	02/Feb	Not started	0%
26	Stakeholder Presentati...	Unassigned	-	02/Feb	02/Feb	Not started	0%
	Testing Phase	Unassigned	-	30/Jan	01/Mar	Not started	0%
28	Unit Testing	Unassigned	-	30/Jan	10/Feb	Not started	0%
29	Integration Testing	Unassigned	-	10/Feb	20/Feb	Not started	0%
30	System Testing	Unassigned	-	20/Feb	01/Mar	Not started	0%
	Evaluation	Unassigned	-	01/Mar	15/Mar	Not started	0%
32	Performance Analysis	Unassigned	-	01/Mar	15/Mar	Not started	0%
33	Stakeholder Feedback ...	Unassigned	-	01/Mar	15/Mar	Not started	0%
	Final Deliverables	Unassigned	-	15/Mar	01/Apr	Not started	0%
35	Minimum Viable Produ...	Unassigned	-	15/Mar	30/Mar	Not started	0%
36	Thesis Preparation	Unassigned	-	15/Mar	01/Apr	Not started	0%
37	Final Submission	Unassigned	-	01/Apr	01/Apr	Not started	0%

References

Chen, G., Yao, Y., Chao, L. S., Liu, X. and Wong, D. F. (2025). SGIC: A Self-Guided Iterative Calibration Framework for RAG. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Vienna, Austria: Association for Computational Linguistics, pp. 28357–28370.

Dhuliawala, S., et al. (2024). Chain-of-Verification Reduces Hallucination in Large Language Models. In: Findings of the Association for Computational Linguistics: ACL 2024. Bangkok, Thailand: Association for Computational Linguistics, pp. 3563–3578.

Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P., Lomeli, M., Hosseini, L. and Jégou, H. (2025). The Faiss library. IEEE Transactions on Big Data.

Ji, Z., Gu, Y., Zhang, W., Lyu, C., Lin, D. and Chen, K. (2024). ANAH: Analytical Annotation of Hallucinations in Large Language Models. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL). pp. 8135–8158.

Johnson, J., Douze, M. and Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), pp. 535-547.

Khasanova Zafar kizi, M. and Suh, Y. (2025). Design and Performance Evaluation of LLM-Based RAG Pipelines for Chatbot Services in International Student Admissions. Electronics, 14(15), 3095.

Kryscinski, W., McCann, B., Xiong, C. and Socher, R. (2020). Evaluating the Factual Consistency of Abstractive Text Summarization. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 9332–9346.

Laban, P., Schnabel, T., Bennett, P. N. and Hearst, M. A. (2022). SummaC: Re-visiting NLI-based Models for Inconsistency Detection in Summarization. Transactions of the Association for Computational Linguistics, 10, pp. 163–177.

Lei, T., et al. (2023). Chain of Natural Language Inference for Reducing Large Language Model

Ungrounded Hallucinations. arXiv:2310.03951 [Pre-print].

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 33, pp. 9459-9474.

Li, J., Cheng, X., Zhao, X., Nie, J.-Y. and Wen, J.-R. (2023). HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP). Singapore: Association for Computational Linguistics, pp. 6449–6464.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis,M., Zettlemoyer, L. and Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692 [Pre-print].

Manakul, P., Liusie, A. and Gales, M. J. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. arXiv:2303.08896 [Pre-print].

Maynez, J., et al. (2020). On Faithfulness and Factuality in Abstractive Summarization. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL). Online: Association for Computational Linguistics, pp. 1906–1919.

Morishita, T., Morio, G., Yamaguchi, A. and Sogawa, Y. (2024). Enhancing reasoning capabilities of llms via principled synthetic logic corpus. In: Advances in Neural Information Processing Systems (NeurIPS).

Perikos, I., Souli, S. and Hatzilygeroudis, I. (2023). A Deep Learning Literature Review for Solving the Natural Language Inference (NLI) Problem. Applied Sciences, 13(4), 2577.

Tao, et al. (2024). When to Trust LLMs: Aligning Confidence with Response Quality and Exploring Applications in RAG. arXiv:2404.17287 [Pre-print].

Wang, S., Wang, X., Mei, J., Xie, Y., Chen, S. and Xiong, W. (2025). Developing a Reliable, Fast, General-Purpose Hallucination Detection and Mitigation Service. In: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track). Albuquerque, New

Mexico: Association for Computational Linguistics, pp. 971–978.

Williams, A., Nangia, N. and Bowman, S. (2018). A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT). New Orleans, Louisiana: Association for Computational Linguistics, pp. 1112–1122.